Artificial Intelligence
CSL7610



Deeksha Varshney

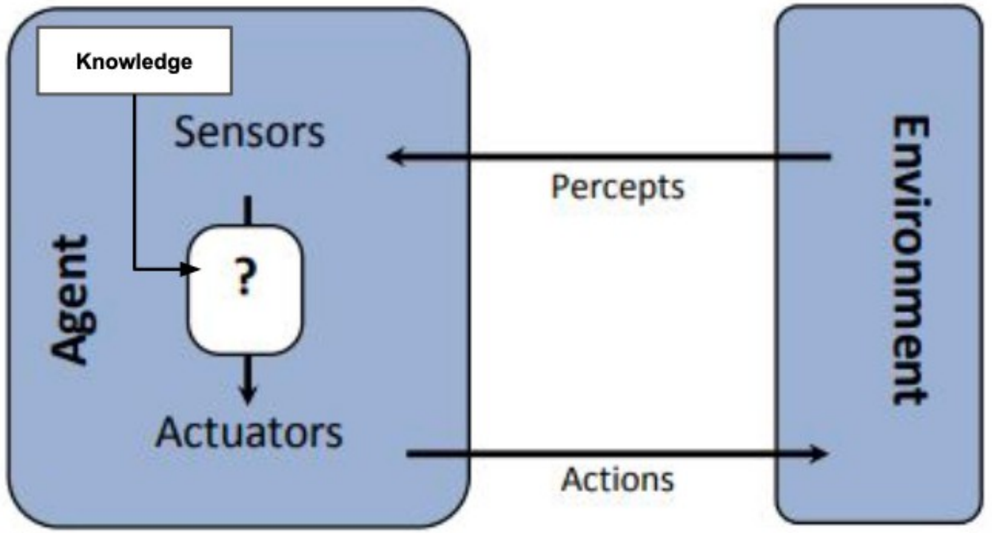
Knowledge Representation

Content is taken from various sources

Knowledge and Intelligence

Role of knowledge in intelligent behavior:

- Without knowledge, the agent cannot choose proper actions.
- Hence, knowledge enables agents to act intelligently in different situations.
- An agent senses events from the environment.
- Based on the perceptions, the agent makes decisions.
- Knowledge supports the agent in deciding what actions to take.
- These decisions drive the agent's actions.



Building Intelligent Agents

- Goal: Create agents that act like humans or show intelligent behavior.
- To achieve this, we need:
  - A way to represent knowledge so the agent can understand it.
  - Mechanisms for using that knowledge to guide actions.

Knowledge Representation & Logic

Link to Previous Topic (Search Problems):

- Earlier, we focused on search (state spaces, paths, goal finding).
- But search alone isn't enough for *intelligent behavior*.
- Example: A maze solver only needs search; but if we want a medical AI agent, it needs *knowledge* about symptoms, diseases, and treatments.

Knowledge Representation (KR):

- KR = how we store information about the world so an agent can *use it*.
- Instead of raw search, the agent now relies on stored **facts + rules**.
- Example: If a patient has *fever AND cough*, then → *flu*.

Why Logic?

- Logic provides a **formal, unambiguous way** to represent facts.
- Unlike natural language (which can be vague), logic is strict, mathematical, and machine-readable.

Knowledge Representation & Logic

- Goal of KR
- Not just *store data*, but store it in a way that allows **inference**.
- Inference = deriving new facts from known ones.
- Example:
  - Fact 1: “All humans are mortal.”
  - Fact 2: “Socrates is human.”
  - Inference: “Socrates is mortal.

Example (Arithmetic Logic):

- Statement: $x \geq 5$
- If $x = 7$, inference → true.
- If $x = 3$, inference → false.
- This shows how a logical statement helps us evaluate truth in different cases.

Knowledge Representation & Logic (contd.)

Why a Language?

- Humans use natural languages (English, Hindi, etc.), but they're often ambiguous.
- Computers/agents need a **formal language** → precise, unambiguous.
- Example: Instead of "John is tall," we write $Tall(John)$ in logic.

Representation vs. Interpretation:

- **Representation** = how knowledge is stored (symbols, rules, logic statements).
- **Interpretation** = how the system understands and uses that stored knowledge.
- Example: Store $Rain \Rightarrow WetRoad$.
  - If it perceives $Rain = True$, the system interprets this and infers → $WetRoad = True$.

Knowledge Representation & Logic (contd.)

Inference Mechanism (The Brain of the Agent):

- Works like human reasoning: takes **facts + rules**, draws **conclusions**.
- Example:
  - Perception: "Temperature = High."
  - Knowledge: "If Temperature is High, then TurnOn(Fan)."
  - Inference: Agent decides → Turn on the fan.

Why Both are Needed:

- A **language alone** (e.g., just storing rules) is useless if the agent cannot use it.
- An **inference mechanism alone** is useless without knowledge to process.
- Together, they form the basis of intelligent decision-making.

Analogy

- **Language = Dictionary/Grammar** (words + rules of expression).
- **Inference Mechanism = Human Brain** (uses dictionary to understand meaning, make decisions).
- Without both, communication → and intelligence → fail.

Logic in AI

Types of Logic (brief):

- **Propositional Logic (PL):** Deals with simple true/false statements.
- **First-Order Logic (FOL):** Adds objects, relations, quantifiers (∀, ∃).
  - Example: “All humans are mortal” = ∀x Human(x) → Mortal(x).
- **Modal Logic:** Deals with necessity, possibility (used in reasoning about belief, time, etc.).

Knowledge Base

What is a Knowledge Base?

- Think of it as the **memory** of the agent.
- Just like humans have stored facts (e.g., “Water boils at 100°C”), agents store sentences in the KB.

Sentences in the KB:

- Must follow **syntax rules** of a formal logic language.
- Example:
  - x + y = 4 → valid (follows arithmetic/logic rules).
  - x4y+= → invalid (nonsense string, not interpretable).

Knowledge Base

Syntax vs. Semantics:

- **Syntax** = the *form* of the sentence (rules for valid structure).
- **Semantics** = the *meaning/truth* of the sentence in a world.
- Example:
  - Syntax: x + y = 4 → valid sentence.
  - Semantics: True if values satisfy it (e.g., x=2, y=2). False otherwise.

True/False in Logic:

- In classical logic, a sentence cannot be "maybe true."
- It is always **true OR false** in a given world (like binary values in computing).
- Example:
  - World 1: x=2, y=2 → x + y = 4 = True.
  - World 2: x=1, y=1 → x + y = 4 = False.

Propositional Logic (PL)

What is a Proposition?

- A proposition is just a **statement about the world**.
- Example:
  - “It is raining.” (True/False) → proposition.
  - “The sky is blue.” (True/False) → proposition.
- Key: it must be **definitely true or false**, not vague.
  - “Is it raining heavily?” → not a proposition (too fuzzy).

Atomic Sentences

- Simplest form → represented by a symbol like **P, Q, R**.
- Example:
  - P: “It is raining.”
  - Q: “The ground is wet.”

Propositional Logic (PL)

Complex Sentences

- Combine atomic propositions with **logical connectives**:
  - **¬P** → "It is not raining."
  - **P ∧ Q** → "It is raining AND the ground is wet."
  - **P ∨ Q** → "It is raining OR the ground is wet."
  - **P → Q** → "If it is raining, then the ground is wet."

Propositional Logic: Connectives

Negation (¬P)

- Flips truth value.
- Example:
  - P = “It is raining.”
  - ¬P = “It is not raining.”

Conjunction (P ∧ Q)

- True **only if both P and Q are true**.
- Example:
  - P = “It is raining.”
  - Q = “The ground is wet.”
  - P ∧ Q = “It is raining AND the ground is wet.”

Propositional Logic: Connectives

Disjunction ($P \vee Q$)

- True if **at least one** is true.
- Example:

  - P = “I will go by bus.”
  - Q = “I will go by train.”
  - $P \vee Q$ = “I will go by bus OR train (or both).”

Propositional Logic: Connectives

Implication (P → Q)

- "If P then Q."
- Only **false** when P is true but Q is false.
- Example:
  - P = "It rains."
  - Q = "The ground is wet."
  - P → Q = "If it rains, then the ground is wet."

Propositional Logic: Connectives

Biconditional ($P \Leftrightarrow Q$)

- True when **both are same** (both true or both false).
- Example:
  - P = “The light is on.”
  - Q = “The switch is up.”
  - $P \Leftrightarrow Q$ = “The light is on if and only if the switch is up.”

Propositional Logic (PL): Syntactical Elements

Propositional Symbols

- Represent **basic facts/statements** about the world.
- Example:
  - P = “It is raining.”
  - Q = “The ground is wet.”
- Each is either **True** or **False** at any moment.

Logical Operators

- Used to **combine propositions** into more complex sentences.
- Example:
  - $P \wedge Q \rightarrow$ “It is raining AND the ground is wet.”
  - $\neg P \rightarrow$ “It is NOT raining.”

Propositional Logic (PL): Syntactical Elements

Parentheses (Grouping)

- Needed when combining multiple operators.
- Example:

◦ $(P \wedge Q) \vee R \neq P \wedge (Q \vee R)$.
◦ First means: either (P and Q) or R.
◦ Second means: P and (Q or R).

Propositional Logic (PL): Logical Sentences

- Each **symbol** (proposition or constant) is a **sentence**.
- A **sentence** always has a **truth value** (True or False).
- **Sentences = Well-formed formulae (WFF).**
- A **WFF** is built using:
  - Propositional variables (P, Q, R, …)
  - Logical connectives (∧, ∨, ¬, →, ⇔)
  - Parentheses for grouping.

Examples:

- (P ∧ Q) ∨ ¬R → valid WFF.
- (P ∧) Q ∨ ¬R → invalid (syntax error).
- TRUE, (P), P ∧ Q, P ∨ Q, ¬P, P → Q, P ⇔Q, (P ∨ Q) → R → valid WFFs.

Implication (→)

P → Q

If **P is True**, then **Q must be True**.
P is a **sufficient condition** (but not necessary) for Q.
Example:

$P \rightarrow Q \equiv \neg P \vee Q$

- **P = "It rains."**
- **Q = "The roads are wet."**
- P → Q = "If it rains, then the roads are wet."

But the reverse (Q → P = "If the roads are wet, then it rains") is **not necessarily true**.

Roads can be wet from other causes (sprinklers, cleaning, flooding).

Biconditional (⇔)

P ⇔Q

Means:

- If **P is True**, then **Q is True**.
- AND if **Q is True**, then **P is True**.

Example:

- "Two sides of a triangle are equal **if and only if** two base angles are equal."
- P = "Two sides are equal."
- Q = "Two base angles are equal."
- Works both ways:
  - P → Q (equal sides → equal base angles).
  - Q → P (equal base angles → equal sides).

Equivalence can be written as:

- **(P → Q) ∧ (Q → P)**

Operator Precedence

Precedence order (highest → lowest):

1. ¬ **(NOT)**
2. ∧ **(AND)**
3. ∨ **(OR)**
4. → **(Implication)**
5. ⇔**(Biconditional)**

Example:

- Formula: $P \vee Q \wedge R \Leftrightarrow Q \Rightarrow \neg R$
- Without rules, it can be parsed in **different ways**.

Rules:

- → **and** ⇔**associate to the right**:
  - $P \Rightarrow Q \Rightarrow R = P \Rightarrow (Q \Rightarrow R)$
  - $P \Leftrightarrow Q \Leftrightarrow R = P \Leftrightarrow (Q \Leftrightarrow R)$
- ∧ **and** ∨ **associate to the left**:
  - $(P \wedge Q) \wedge R$
  - $(P \vee Q) \vee R$

Definitions

A **premise** is a statement (or proposition) that provides support or evidence in an argument.

- In logic, an **argument** is made up of **premises** (the reasons) and a **conclusion** (what those reasons are supposed to show).
- The premises are assumed to be true for the purpose of evaluating whether the argument is **valid**.
- They don't have to be factually true - they just serve as the **starting points** of reasoning.

Example:
Premise 1: All humans are mortal.
Premise 2: Socrates is a human.
Conclusion: Therefore, Socrates is mortal.

Truth Value Examples

- If P = True, Q = True:
  - $P \rightarrow Q$.
  - $(\neg P \vee Q) \rightarrow Q$.
  - $(\neg P \vee Q) \rightarrow P$.
  - $P \vee \neg P \rightarrow T$.

Truth Value Examples

1. $P \Rightarrow Q$
   - If P is True, then Q is True.
   - Result: True
2. $(\neg P \vee Q) \Rightarrow Q$
   - $\neg P = F$, so $\neg P \vee Q = T$.
   - So $T \Rightarrow Q = T \Rightarrow T = T$.
   - Result: True
3. $(\neg P \vee Q) \Rightarrow P$
   - $\neg P = F$, so $\neg P \vee Q = T$.
   - So $T \Rightarrow P = T \Rightarrow T = T$.
   - Result: True
4. $(P \vee \neg P) \Rightarrow T$
   - $P \vee \neg P = T \vee F = T$.
   - So $T \Rightarrow T = T$.
   - Result: True

Validity of a Sentence

- A **propositional sentence** is **valid** if it is **true under all possible interpretations**.
- **Tautology:** a formula that is **always true**, no matter the truth values of its propositional variables.

Example:

- $P \vee \neg P$ (Law of the Excluded Middle)
  - If $P = \text{True}$, then $P \vee \neg P = \text{True}$
  - If $P = \text{False}$, then $\neg P = \text{True}$, so $P \vee \neg P = \text{True}$

Key Point:

- A **tautology** is **universally true**.
- Validity is fundamental in logic for reasoning and proving arguments.

An interpretation is simply an assignment of truth values to the propositions.

PL: English to Logic

Practice Questions

1. Express the following statements in propositional logic:
   - “The light is on.”
   - “The fan is off.”
2. Translate into PL:
   - “If it is raining and the ground is muddy, then the match will be canceled.”
3. Using propositions:
   - R = “It is raining”
   - M = “The ground is muddy”
   - C = “The match will be canceled”
   - Write PL formulas for the above statements.
4. Combine statements using logical connectives:
   - “The light is on OR the fan is off.”
   - “If the light is on AND the fan is off, then the room is uncomfortable.”

PL: English to Logic

1. "The light is on." → L
2. "The fan is off." → F
3. Conditional statement:
   - "If it is raining and the ground is muddy, then the match will be canceled." → $R \wedge M \Rightarrow C$
4. Using given propositions:
   - "The light is on OR the fan is off." → $L \vee F$
   - "If the light is on AND the fan is off, then the room is uncomfortable." → $(L \wedge F) \Rightarrow U$
     - (Assume U = "Room is uncomfortable")

Deriving Truth Values for Compound Sentences

Definition: Shows the truth value of a propositional formula for all possible values of its atomic propositions.

Purpose:

- Derive correctness or validity of any propositional sentence.

P	Q	¬P	P∧Q	P∨Q
T	T	F	T	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	F

Equivalence of Formulas

- Two formulas $F1$ and $F2$ are **equivalent** if they have the same truth value for all interpretations
- **Example:** $P \vee P \equiv P$
- **Important:** Always **mention the equivalence used** when simplifying formulas.

Equivalence of Formulas

- **Law of Double Negation** $\neg\neg\phi \equiv \phi$
- Negating a statement twice brings you back to the original statement.

- **Identity Laws** $\phi \wedge T \equiv \phi$ $\phi \vee F \equiv \phi$
  - Conjunction with `True` doesn't change the statement.
  - Disjunction with `False` doesn't change the statement.

- **Domination Laws** $\phi \vee T \equiv T$ $\phi \wedge F \equiv F$
  - Disjunction with `True` always gives `True`.
  - Conjunction with `False` always gives `False`.

Take **any formula ϕ, AND it with True, and you get the same formula.**

Equivalence of Formulas

- **Idempotent Laws** $\phi \vee \phi \equiv \phi \qquad \phi \wedge \phi \equiv \phi$
  - Repeating a statement with AND or OR doesn't change its value.

- **Negation Laws** $\phi \wedge \neg\phi \equiv F \qquad \phi \vee \neg\phi \equiv T$
  - A statement AND its negation is always `False`.
  - A statement OR its negation is always `True`.

- **Absorption Laws** $\phi_1 \wedge (\phi_1 \vee \phi_2) \equiv \phi_1 \qquad \phi_1 \vee (\phi_1 \wedge \phi_2) \equiv \phi_1$
  - One statement "absorbs" the combination with the other, simplifying expressions.

CNF vs DNF

- **CNF, or Conjunctive Normal Form,** means the formula is written as an AND of clauses, and each clause is an OR of literals.
- **For example, here we have** $(P \vee Q) \wedge (R \vee \neg S)$**.**

  You can think of CNF like a set of rules - all the rules must hold true, but within each rule, you have some flexibility.

CNF vs DNF

- **DNF, or Disjunctive Normal Form,** is the opposite structure: it's an OR of terms, and each term is an AND of literals.
- **For example, here we have $(P \wedge R) \vee (\neg Q \wedge S)$.**
  - DNF is more like a list of scenarios - if any one scenario holds, the whole formula is true.

De Morgan's Theorem

"De Morgan's Theorems tell us how negation distributes over AND and OR.

- First Law: $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
- Second Law: $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
- If we negate an **AND**, it becomes an **OR** of the negations.
- If we negate an **OR**, it becomes an **AND** of the negations.

De Morgan's Theorem

- The truth table here shows the second law: **¬(P∨Q) is equivalent to ¬P∧¬Q**.
- We can see that for every possible truth value of P and Q, the two expressions give exactly the same result.
- This equivalence is extremely useful in logic simplification, digital circuits, and computer science, because it allows us to rewrite formulas in different but equivalent ways."

P	**Q**	**P∨Q**	**¬(P∨Q)**	**¬P**	**¬Q**	**¬P∧¬Q**
T	T	T	F	F	F	F
T	F	T	F	F	T	F
F	T	T	F	T	F	F
F	F	F	T	T	T	T

Equivalences

Prove that the following formulas are equivalent:

- $\neg(P \vee (\neg P \wedge Q))$ and $\neg P \wedge \neg Q$
- $\neg(P \rightarrow Q)$ and $P \wedge \neg Q$

Equivalences

1) $\neg\big(P \vee (\neg P \wedge Q)\big) \equiv \neg P \wedge \neg Q$

Algebraic proof (use De Morgan, distributivity, contradiction):

$$
\begin{aligned}
\neg\big(P \vee (\neg P \wedge Q)\big) &= \neg P \wedge \neg(\neg P \wedge Q) && (\text{De Morgan: } \neg(A \vee B) = \neg A \wedge \neg B)\\
&= \neg P \wedge (P \vee \neg Q) && (\text{De Morgan: } \neg(\neg P \wedge Q) = P \vee \neg Q)\\
&= (\neg P \wedge P) \ \vee \ (\neg P \wedge \neg Q) && (\text{distribute } \neg P \text{ over } (P \vee \neg Q))\\
&= \text{False} \ \vee \ (\neg P \wedge \neg Q) && (\neg P \wedge P \text{ is contradiction})\\
&= \neg P \wedge \neg Q.
\end{aligned}
$$

So the left-hand formula simplifies exactly to $\neg P \wedge \neg Q$.

Equivalences

2) $\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$

Recall the standard equivalence $P \Rightarrow Q \equiv \neg P \vee Q$. Then

$$\begin{aligned} \neg(P \Rightarrow Q) &= \neg(\neg P \vee Q) \\ &= P \wedge \neg Q \qquad \text{(by De Morgan)}. \end{aligned}$$

Variations of Implications

- Converse of an implication $P \rightarrow Q$ is $Q \rightarrow P$
- Inverse of an implication $P \rightarrow Q$ is $\neg P \rightarrow \neg Q$
- Contrapositive of an implication: $P \rightarrow Q$ is $\neg Q \rightarrow \neg P$
- Compare them using Truth Tables and also using the equivalences

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Figure 7.11 Standard logical equivalences. The symbols α, β, and γ stand for arbitrary sentences of propositional logic.

Models

1.Intuition

- *In logic, we imagine different possible worlds. Each world is like a consistent snapshot of reality."*
- Example: One world might be *'Giants won, Reds lost'*, another might be *'Giants won, Reds won'*.

2.Models in Plain Language:

- A **model** is just one of these possible worlds.
- If a sentence is true in that world, we say the model "satisfies" the sentence.
- Example: In the world where *Giants won*, the sentence *'Giants won'* is satisfied.

Models

1.Connecting Models to Entailment:

- M(s) = set of all worlds where sentence *s* is true.
- KB entails *s* (KB ⊨ s) if every world where the KB is true is also a world where *s* is true.
- In other words, if you accept the KB, you can't avoid accepting *s*.

2.Work Through the Example:

- KB = {Giants won, Reds won}.
- M(KB) = set of worlds where both are true.
- Let s = "Giants won"
- M(s) = set of worlds where "Giants won" is true.
- Since, M(KB) requires both Giants and Reds winning, any world in M(KB) is certainly in M(s) (because Giants won is part of KB).
- **Clearly, M(KB) is a subset of M(s). So KB ⊨ s.**

Models

If it were $M(s) \subseteq M(KB)$, then that would mean:

- "Every world where Giants won is also a world where Giants **and** Reds won."
- That's not true, because Giants might have won but Reds didn't (so those worlds would be in $M(s)$ but not in $M(KB)$).

KB $\models$ s means: "In every world where KB is true, s is also true."

Giants won	Reds won	World?
T	T	KB true
T	F	KB false
F	T	KB false
F	F	KB false

M(KB) = {T,T}

M(s) = {T,T; T,F}

Entailment

- Set of sentences S true under interpretations I
  - If another sentence s1 is true in all interpretations in I, then S entails s1
  - This is logical entailment
- $S \models s1$
- s1 is a logical consequence of S
- Equivalent to: $S \rightarrow s1$ is valid
- Entailment means that if a knowledge base S is true, then another sentence s1 must also be true in the same interpretations. We write this as $S \models s1$, equivalent to $S \rightarrow s1$ being valid.

Entailment Illustration

Let's define two propositional symbols:

- **G**: "Giants won"
- **R**: "Reds won"

Knowledge base S:

1. G
2. $G \rightarrow R$ (If Giants won, then Reds won)

So:

$$S = \{G,\ (G \rightarrow R)\}$$

Entailment Illustration

We have two variables G and R. So 4 possible interpretations:

G	R	$G \rightarrow R$	S true?
T	T	T	Yes
T	F	F	No
F	T	T	No (because G is false)
F	F	T	No (because G is false)

So only **(G=T, R=T)** satisfies both formulas in S.

Let's call this set of models $M(S) = \{(T, T)\}$.

Entailment Illustration

Step 2 — Consider a new sentence $s_1 = R$

- Where is R true?

  In (T,T) and (F,T).
- So $M(R) = \{(T,T), (F,T)\}$.

Step 3 — Check entailment

Entailment says:

$$S \models R \quad \text{iff} \quad M(S) \subseteq M(R)$$

We see:

$$\{(T,T)\} \subseteq \{(T,T), (F,T)\}$$

So this is **true**.
Therefore, S ⊨ R.

Every interpretation that makes all of **S** true also makes **R** true.

Satisfiability

- A sentence is satisfiable if there exists an interpretation where it's true.
- A sentence is unsatisfiable if no interpretation makes it True.

- Example: $P \wedge \neg P$ is unsatisfiable (always false) because it requires P to be true and false simultaneously.

Example

Example: P∧Q

P	**Q**	P ∧ Q
T	T	**T**
T	F	F
F	T	F
F	F	F

There is **one interpretation** where it is true: P=T,Q=T, so P^Q is satisfiable.

PROPOSITIONAL THEOREM PROVING

Reasoning Example: Modus Ponens

- If it is the month of July then it rains
- It is the month of July
- Conclude: It rains

- Symbols:
- P: It is the month of July
- Q: It rains
- R: P→Q

  - Inference:
  - P→Q,
  - P
  - _ _ _ _
  - Q

- Rule: Modus Ponens

Basic Inference Rules

- $\frac{P \wedge Q}{P}$ [And Elimination] [Explanation: If P and Q both are true, then P will also be True]

- $\frac{P \quad Q}{P \wedge Q}$ [And Introduction] [Explanation: If we know that P is True and Q is True, then P and Q will also be True]

- $\frac{P}{P \vee Q}$ [Or Introduction] [Explanation: If P is true, then P∨Q will also be True]

- $\frac{P \vee Q \quad P \Rightarrow R \quad Q \Rightarrow R}{R}$ [Or Elimination] [Explanation: If P or Q is true, and P⇒R and Q⇒R are True, then R will also be True]

More Inference Rules

- Chain Rule (Hypothetical Syllogism): If P implies Q and Q implies R, then P implies R.
  - $P \rightarrow Q$,
  - $Q \rightarrow R$
  - ————
  - $P \rightarrow R$
- Modus Tollens: If P implies Q, but Q is false, then P must also be false.
  - $P \rightarrow Q$,
  - $\neg Q$
  - ————
  - $\neg P$

Proof methods

Model checking

- – Truth table enumeration (sound and complete for propositional logic)
  - Show that all interpretations in which the left hand side of the rule is true, the right hand side is also true

P	H	$P \vee H$	$(P \vee H) \wedge \neg H$	$((P \vee H) \wedge \neg H) \Rightarrow P$
False	*False*	*False*	*False*	*True*
False	*True*	*True*	*False*	*True*
True	*False*	*True*	*True*	*True*
True	*True*	*True*	*False*	*True*

– Application of inference rules

- Sound generation of new sentences from old
  Proof = a sequence of inference rule applications
  Can use inference rules as operators in a standard search algorithm

Soundness

An inference algorithm is **sound** if it never derives something that does not logically follow.

Formally:

$$KB \vdash_i \alpha \quad \Rightarrow \quad KB \models \alpha$$

Completeness

Completeness is the opposite direction.

An inference algorithm is **complete** if it can derive **every sentence that is actually entailed**.

Formally:

$$KB \models \alpha \quad \Rightarrow \quad KB \vdash_i \alpha$$

Property	Meaning
Sound	Never derives an incorrect conclusion
Complete	Can derive every correct/entailed conclusion

Inference Technique 1: Enumeration

- Technique 1: Truth Table Enumeration
- KB: $A \vee C$, $B \vee \neg C$
- Query $\beta = A \vee B$
- Show $KB \models \beta$
- Build a complete truth table to check

Truth Table for KB ⊨ β

We use atoms A, B, C. KB = $\{A \vee C,\ B \vee \neg C\}$. Query $\beta = A \vee B$.

Build the full truth table (8 rows). Mark where each clause of KB is true, whether KB (both clauses) is true, and whether β is true.

row	A	B	C	A ∨ C	B ∨ ¬ C	KB true?	A ∨ B
1	T	T	T	T	T	**T**	T
2	T	T	F	T	T	**T**	T
3	T	F	T	T	F	F	T
4	T	F	F	T	T	**T**	T
5	F	T	T	T	T	**T**	T
6	F	T	F	F	T	F	T
7	F	F	T	T	F	F	F
8	F	F	F	F	T	F	F

Truth Table for KB ⊨ β

row	A	B	C	A ∨ C	B ∨ ¬ C	KB true?	A ∨ B
1	T	T	T	T	T	**T**	T
2	T	T	F	T	T	**T**	T
3	T	F	T	T	F	F	T
4	T	F	F	T	T	**T**	T
5	F	T	T	T	T	**T**	T
6	F	T	F	F	T	F	T
7	F	F	T	T	F	F	F
8	F	F	F	F	T	F	F

Now look at the rows where **KB is true**: rows **1, 2, 4, 5**. In every one of those rows the query $A \vee B$ is **true** (T).

Therefore every model of KB is also a model of β: $M(KB) \subseteq M(\beta)$. By the definition of entailment,

$$\mathbf{KB} \models (A \vee B).$$

(As a quick extra note: this is exactly the result you can also get by **resolution** on C: from $A \vee C$ and $B \vee \neg C$ resolve C to obtain $A \vee B$.)

Remarks on Enumeration

- Enumeration requires full truth table
- 100 propositions → 2^100 rows
- Exponential time, very slow

Inference Technique 2: **Natural Deduction**

- Uses inference rules and equivalences
- Build a proof: sequence of steps from KB to β (query)
- More efficient than enumeration
- Natural deduction avoids full enumeration. Instead, it systematically applies inference rules to derive the query step by step.

Proof Example

Does R logically follow (is it entailed) from the KB?

- KB:
  - $(P \wedge Q) \rightarrow R$
  - $(S \wedge T) \rightarrow Q$
  - S
  - T
  - P
  - Query: R

- Proof:

1. S	[Premise]
2. T	[Premise]
3. P	[Premise]
4. $S \wedge T$	[And Intro]
5. $(S \wedge T) \rightarrow Q$	[Premise]
6. Q	[Modus Ponens]
7. $P \wedge Q$	[And Intro]
8. $(P \wedge Q) \rightarrow R$	[Premise]
9. R	[Modus Ponens]

Practice Questions

Let P, Q, R, and S be arbitrary propositions. Which of the following arguments are valid?

$$1) P \rightarrow Q, \neg P \vdash \neg Q,$$

$$3) P \vee R, P \rightarrow Q, R \rightarrow S \vdash S \vee Q,$$

$$4) \vdash (P \rightarrow Q) \vee (Q \rightarrow P)$$

Validity means: **if all the premises are true, the conclusion must also be true.**

Practice Questions

A father and his son met with an accident. The father died instantly; the son was taken to the hospital. The surgeon said, "I cannot operate on this boy. He is my son." Is the set of statements describing this situation satisfiable?

A) Yes - it is satisfiable if "the surgeon" is the boy's mother.

B) No - it leads to contradiction.

C) Only if the surgeon is not human.

D) Cannot be determined.

Satisfiable = there exists at least one model/interpretation where all statements are TRUE.

Practice Questions

Given the Knowledge Base (KB):

$$(1) P \Rightarrow Q, \quad (2) R \Rightarrow S, \quad (3) (Q \vee S) \Rightarrow T, \quad (4) T \Rightarrow U,$$

$$(5) P$$

Which of the following is entailed by the KB?

A) $P \wedge R$
B) $T \wedge Q$
C) U
D) $\neg S$

Practice Questions

Given the Knowledge Base (KB):

$(1) A \vee B, \quad (2) \neg A \vee C, \quad (3) C \Rightarrow (D \wedge E), \quad (4) \neg B$

Which of the following is entailed by the KB?

A) $A \wedge C$ **B)** $\neg A \wedge \neg C$ **C)** $B \vee D$ **D)** E

Monotonicity

- In propositional logic: once true, always true
- Adding new sentences does not remove old conclusions
- Called the Monotonicity Property

Scenario:

- You know for sure:
  - "All humans are mortal."
  - "Socrates is a human."

Conclusion:

- "Socrates is mortal."

Now, suppose you **add more facts**, like:

- "The sky is blue."
- "Socrates lived in Athens."

These **new facts do not change** the earlier conclusion. "Socrates is mortal" **remains true** no matter how many unrelated facts you add.

If a knowledge base KB entails a conclusion α:

$$KB \models \alpha$$

then for **any new sentence** γ,

$$KB \cup \{\gamma\} \models \alpha.$$

An important property is monotonicity. Once you've proved a fact, adding more knowledge cannot make it false. This guarantees consistency in reasoning.

Clause Form

- Literal: a proposition or its negation
  - P, ¬P
- Clause: disjunction of literals
  - e.g., P∨Q∨¬R
- Clausal form useful for inference

Why Clausal Form is Useful

- Resolution inference works only on clauses.
- So, when proving entailment using resolution, we:
  1. Convert all sentences into clausal form (CNF),
  2. Then apply resolution on pairs of clauses until either:
     - we derive an empty clause (contradiction → proof complete), or
     - no new clauses can be produced.

Converting to Clausal Form

- Sentence: $\neg(A \rightarrow B) \vee (C \rightarrow A)$
  1. Eliminate implications:
     - $\neg(\neg A \vee B) \vee (\neg C \vee A)$
  2. Apply De Morgan:
     - $(A \wedge \neg B) \vee (\neg C \vee A)$
  3. CNF:
     - $(A \vee \neg C) \wedge (\neg B \vee \neg C \vee A)$
  4. Clauses:
     - $(A \vee \neg C)$, $(\neg B \vee \neg C \vee A)$

Clausal Form allows us to apply Inference by Resolution

Conversion Rules

- Implication elimination:
  - $A \rightarrow B \rightarrow \neg A \vee B$
  - $A \Leftrightarrow B \rightarrow (\neg A \vee B) \wedge (\neg B \vee A)$
- De Morgan's laws:
  - $\neg(A \vee B) \rightarrow \neg A \wedge \neg B$
  - $\neg(A \wedge B) \rightarrow \neg A \vee \neg B$
- Distribution:
  - $(A \wedge B) \vee C \rightarrow (A \vee C) \wedge (B \vee C)$
  - $(A \vee B) \wedge C \rightarrow (A \vee B) \wedge C$

Note: Always mention the equivalences/laws being used - distributing.. or implication elimination or ……

Resolution

- If we have two clauses:
- $(x \vee S1)$ and $(\neg x \vee S2)$
- Then we get: $S1 \vee S2$ (resolvent).
  - x is “resolved upon”.
- Equivalent to proving:
  - $(x \vee S1) \wedge (\neg x \vee S2) \rightarrow (S1 \vee S2)$ is a tautology.
- **Explanation:**
  - If one clause says “either x or S1 is true” and another says “either not x or S2 is true”, then we can eliminate x and conclude “S1 or S2 is true”.
  - This process of eliminating a literal and its complement is called **resolution**.
  - Resolution is powerful because it allows us to simplify knowledge bases and reason automatically.

Resolution

- Clauses:
  - $P \vee Q \vee R$
  - $\neg Q \vee S \vee T$
- Resolvent:
  - $P \vee R \vee S \vee T$
- If no symbols left after resolving → Empty clause ($\perp$ or False).

Here's a concrete example:

- One clause says at least one of {P, Q, R} is true.
- Another says either $\neg Q$ or {S, T} is true.
- Since Q and $\neg Q$ cancel each other, the resolvent is $P \vee R \vee S \vee T$.
- If after repeated resolution we get an empty clause, it means we reached a **contradiction** → the KB entails the conclusion.

Resolution Refutation – Method

- To prove KB $\models \beta$:
  - Add $\neg\beta$ to KB.
  - Convert KB to clause form.
  - Apply resolution repeatedly.
  - If empty clause ($\perp$) is derived $\rightarrow$ KB $\models \beta$.
- Proof by contradiction.

This technique is called **Resolution Refutation**.

- Instead of proving β directly, we assume $\neg\beta$.
- If adding $\neg\beta$ leads to a contradiction, then β must be true.
- This method is complete: if something is entailed, resolution refutation can find it.

Inference Technique 3: Inference by Resolution Refutation

Knowledge Base (KB):

(P V Q) → R

R → S

P V T

¬S V U

Query:

U?

1. $(P \vee Q) \rightarrow R \equiv \neg(P \vee Q) \vee R \equiv (\neg P \wedge \neg Q) \vee R \equiv (\neg P \vee R) \wedge (\neg Q \vee R)$
   - Clauses: ¬P ∨ R, ¬Q ∨ R
2. $R \rightarrow S \equiv \neg R \vee S$
3. $P \vee T$ → already in CNF
4. ¬S ∨ U → already in CNF

Inference Technique 3: Inference by Resolution Refutation

Knowledge Base (KB):

(P V Q) → R

R → S

P V T

¬S V U

Query:

U?

1. Resolve ¬S ∨ U (5) with ¬U (6) → ¬S
2. Resolve ¬R ∨ S (3) with ¬S → ¬R
3. Resolve ¬P ∨ R (1) with ¬R → ¬P
4. Resolve P ∨ T (4) with ¬P → T

Does not entail.

Practice Questions

The Knowledge Base is: $KB = \{A \wedge (A \Rightarrow B), C \vee \neg B\}$.

Applying resolution to $KB \wedge \neg(B \Rightarrow C)$, Is the conclusion $B \Rightarrow C$ entailed by the KB?

A) Yes, because resolution derives the empty clause $\perp$

B) No, because resolution derives a non-empty clause that can satisfy the premises

C) Yes, because B is the only possible conclusion from the first two premises

D) No, because $C \vee \neg B$ is inconsistent with $\neg(B \Rightarrow C)$

Practice Questions

Given the clauses for KB $\wedge \neg R$:

$(1) P \vee Q, \quad (2) \neg P \vee S, \quad (3) \neg Q \vee R, \quad (4) \neg S, \quad (5) \neg R$

Which sequence of resolution steps successfully derives the empty clause ($\perp$)?

A) $(P \vee Q, \neg P \vee S) \rightarrow (Q \vee S), (Q \vee S, \neg Q \vee R) \rightarrow R, (\neg R, R) \rightarrow \perp$

B) $(\neg R, \neg Q \vee R) \rightarrow \neg Q, (\neg Q, P \vee Q) \rightarrow P, (P, \neg P \vee S) \rightarrow S, (S, \neg S) \rightarrow \perp$

C) $(\neg R, \neg Q \vee R) \rightarrow \neg Q \vee R, (\neg Q \vee R, \neg S) \rightarrow \perp$

D) $(P \vee Q, \neg P \vee S) \rightarrow (Q \vee S), (\neg Q \vee R, \neg S) \rightarrow R, (\neg R, R) \rightarrow \perp$

Question

If the unicorn is mythical, then it is immortal, but if it is not mythical, it is a mammal. If the unicorn is either immortal or a mammal, then it is horned.

Prove: the unicorn is horned

Solution

Step 1: Define propositions

Let:

M: The unicorn is mythical.
I: The unicorn is immortal.
A: The unicorn is a mammal.
H: The unicorn is horned.

Given statements:

1. If the unicorn is mythical, then it is immortal: $M \rightarrow I$
2. If the unicorn is not mythical, then it is a mammal: $\neg M \rightarrow A$
3. If the unicorn is either immortal or a mammal, then it is horned: $(I \vee A) \rightarrow H$
4. We want to **prove** H

Solution

Clauses:

1. $\neg M \lor I$ (from $M \rightarrow I$)
2. $M \lor A$ (from $\neg M \rightarrow A$)
3. $\neg I \lor \neg A \lor H$ (from $(I \lor A) \rightarrow H$)
4. $\neg H$ (negated goal for proof by contradiction)

Resolution steps given:

1. Resolve $\neg H$ with $\neg I \lor \neg A \lor H \rightarrow$ get $\neg I \lor \neg A$
2. Resolve $\neg I \lor \neg A$ with $M \lor A \rightarrow$ get $\neg I \lor M$
3. Resolve $\neg I \lor M$ with $\neg M \lor I \rightarrow$ get empty clause

Conclusion:

- Empty clause obtained → contradiction → original goal H (unicorn is horned) is **proven true**.

Limitations of Propositional Logic

- Propositional Logic cannot represent general facts.
- Example:
  - KB: All dogs are faithful.
  - Tommy is a dog.
  - Query: Tommy is faithful.
- PL fails: $P \wedge T \not\models R$

This is the main limitation of propositional logic:

- It treats every fact as a standalone proposition.
- It cannot capture general statements like “all dogs are faithful”.
- We cannot infer that “Tommy is faithful” even if we know Tommy is a dog.
- This is why we need **First Order Logic (FOL).**

Limitations of Propositional Logic

Example

Imagine we write:

- **P** = “All dogs are faithful”
- **T** = “Tommy is a dog”

In PL:

- **P** is just a **symbol** (like a light switch: on/off)
- **T** is just another symbol

PL does **not** understand that “dog” appears inside both P and T.

It cannot see that these two statements are **related by meaning**.

So, even if P and T are both true, PL **cannot conclude** “Tommy is faithful.”

Key point

- PL = **black boxes** (whole sentences as single units)
- FOL = **opens the boxes** and sees their **structure** (like "Dog(Tommy)" and "Faithful(Tommy)")

First-Order Predicate Calculus

- Propositional Logic uses only propositions (symbols representing facts), only possible values are True and False

FOL: Extends propositional logic with structure

Can represent:

- **Objects (atoms):** people, numbers, places, ideas
- **Relations (predicates):** relationships between objects
- **Properties (predicates):** properties of objects
- **Functions:** return objects from other objects
- **Constants:** named specific objects

Relations: father(fred, mary) → Fred is Mary's father
Properties: red(ball) → The ball is red
Functions: father-of(mary) → returns Mary's father
Constants: MARY

Example

Propositional Logic

- Treat the whole statement as a **single proposition**
  - Example:

    MANSOCRATES
- Meaning: “Socrates is a man”
- **Note:** No internal structure; cannot reason about “Socrates” or “man” separately

2. First-Order Predicate Logic

- Uses **predicates** to describe **properties** of objects
  - Example:

    Man(SOCRATES)
- Meaning: “Socrates has the property of being a man”
- **Advantages:** Can combine with other statements, reason about categories, relationships, and objects

Key Components of First-Order Logic

- **Constant Symbols**
  - **Definition:** Capitalized names representing specific objects
  - **Examples:** SOCRATES, MARY
  - **Interpretation:** Maps to **exactly one object** in the world
  - **Function form:** No arguments, e.g., PIETER (A constant symbol behaves like a function that takes **zero arguments** and always returns the same object.)

Key Components of First-Order Logic

- **Predicates**
  - **Definition:** Represent **properties** or **relationships** between objects. Predicates are like functions except that their return type is True or False.
  - **Syntax:** PredicateName(arg1, arg2, …)
  - **Example:**
    - Man(SOCRATES) → “Socrates is a man”
    - Father(FRED, MARY) → “Fred is Mary’s father”
  - **Interpretation:** True/False based on the objects

Key Components of First-Order Logic

- **Functions**
  - **Definition:** Map objects to exactly **one object** in the world,
  - **Syntax:** FunctionName(arg1, arg2, …)
  - **Example:**
    - `FatherOf(MARY)` → returns the object who is Mary's father
    - `Next(3)` → returns 4

Types of Predicates

- **Examples**
  - Dog(Tommy)
  - Dog(x)

Question:

- What's different here?

Explaination:

- Dog(Tommy). → a **predicate with no variables** (called a **ground atomic formula**)

  - like a **proposition**.

- Dog(x) → a **predicate with one variable**, called a **property**.
  - True for some x, false for others.

Analogy:

Think of Dog(x) as a "test" - if you put in Tommy, it says True; if you put in Table, it says False.

Building More Complex Predicates

- **Predicate logic is like algebra for truth**
- $man(x) \Rightarrow mortal(x)$

  Explanation:
- "If x is a man, then x is mortal."

  - $(Dog(x) \land Brown(x))$ → "x is a dog **and** x is brown"
  - $(Student(x) \lor Teacher(x))$ → "x is a student **or** x is a teacher"
  - $\neg Tall(x)$ → "x is **not** tall"

- Predicates can be **combined with logical operators** ($\land$, $\lor$, $\neg$, $\rightarrow$) just like propositions.

Examples with Multiple Variables

Example:

Loves(x,y)
Meaning: "x loves y."

Loves(x,y) ⇒ Respects(x,y) - If x loves y, then x respects y.

Ex. names for x and y to ground it:

- Loves(Alice, Bob)
- Loves(x, Bob) ⇒ Respects(x, Bob)

Definitions

- **Terms**
  - Anything that **identifies an object**
  - **Types of terms:**
    - **Constant:** Named object, e.g., SOCRATES (function with 0 arguments)
    - **Function:** Maps objects to other objects, e.g., `FatherOf(Alex)`

Definitions

- **Atomic Sentences**
  - Predicate applied to **term arguments**
  - Represents a basic fact that is **True or False**
  - **Examples:**
    - Enemies(WilyCoyote,RoadRunner)- WileE.Coyote and the Road Runner are enemies.
    - Married(FatherOf(Alex),MotherOf(Alex)) - The father of Alex is married to the mother of Alex.
- **Predicates express properties or relationships, and the terms are the objects those properties/relations apply to.**

Definitions

- **Literals**
  - **Atomic sentences** or **negated atomic sentences**
  - Examples:
    - Man(Socrates)
    - ¬Man(Peter)

Definitions

- **Connectives**
- Combine sentences into more complex formulas
- Common connectives:
  - Conjunction: ∧ (“and”) → components are **conjuncts**
  - Disjunction: ∨ (“or”) → components are **disjuncts**
  - Implication: →(“if … then …”)
  - Biconditional: ↔(“if and only if”)
  - Negation: ¬ (“not”)

Definitions

5. Quantifiers

- **Universal (∀):** “For all”
  - Example: $\forall x \quad (Man(x) \rightarrow Mortal(x))$ → “All men are mortal”
- **Existential (∃):** “There exists”
  - Example: $\exists x \quad (Cat(x) \wedge Black(x))$ → “There exists a black cat”

Universal Quantifier Exercise

- Write predicate logic for these statements:

  - Mammals drink milk
  - Man is mortal
  - Man is a mammal
  - Tom is a man

In natural language, when we say statements like:

- "Mammals drink milk"
- "Man is mortal"
- "All men are mammals"

…we are talking about **every member of a certain group**, not just one individual.

If we are talking about **one specific individual**, we **do not use** ∀. For example:

- "Tom is a man" → Man(Tom)

  No ∀ here, because we are only talking about Tom, not all humans.

Universal Quantifier Exercise

English	Predicate Logic	Explanation
Mammals drink milk	$\forall x$ (mammal(x) $\rightarrow$ drink(x, Milk))	For every x, if x is a mammal, then x drinks milk
Man is mortal	$\forall x$ (man(x) $\rightarrow$ mortal(x))	For every x, if x is a man, then x is mortal
Man is a mammal	$\forall x$ (man(x) $\rightarrow$ mammal(x))	All men are mammals
Tom is a man	man(Tom)	Tom is a specific man (a **constant**)

Existential Quantifier Exercise

Existential Quantifiers (∃)

Goal: Understand "there exists at least one".

Examples:

- "At least one planet has life on it"

  $\exists x \quad (planet(x) \land haslife(x))$

- "There exists a bird that cannot fly"

  $\exists x \quad (bird(x) \land \neg fly(x))$

Tip:

- **∃ goes with ∧** — because we are saying **"find one x that has both properties"**.
- **∀ goes with →** — because we are saying **"if x has property A then x has property B"**.

Duality of Quantifiers

$$\forall x\, P(x) \equiv \neg \exists x\, \neg P(x)$$

$$\exists x\, P(x) \equiv \neg \forall x\, \neg P(x)$$

"For all x, P(x) is true"
<–> “There does not exist an x such that P(x) is false."
"There exists an x such that P (x) is true"
<–> “It is not true that P(x) is false for all

Universals can be rewritten as negated existentials.

Existentials can be rewritten as negated universals.

Duality of Quantifiers

- **“For all” (∀)** means **every single one** has to follow the rule.
- **“There exists” (∃)** means **at least one** follows the rule.

They are logical opposites because:

- If not *everyone* follows a rule → then at least *someone* breaks it.
- If not *someone* follows a rule → then *no one* follows it, which means everyone breaks it.

Sentences in FOL

Goal: How to build complete logical sentences.

- If $s1$ and $s2$ are sentences, then
  - $\neg s1$, $s1 \wedge s2$, $s1 \vee s2$, $s1 \Rightarrow s2$, $\forall x\ s1$, $\exists x\ s1$ are also sentences
- This is just like propositional logic - **but using predicates**

Practice examples:

- Some dogs bark

  $\exists x\ (dog(x) \wedge bark(x))$
- All dogs have four legs

  $\forall x\ (dog(x) \Rightarrow legs(x,4))$
- All barking dogs are irritating

  $\forall x\ ((dog(x) \wedge bark(x)) \Rightarrow irritating(x))$
- No dogs purr

  $\forall x\ (dog(x) \Rightarrow \neg purr(x))$ or $\neg \exists x\ (dog(x) \wedge purr(x))$
- Students are people who are enrolled in courses

  $\forall x\ (student(x) \Rightarrow (people(x) \wedge enrolled(x)))$

What Counts as a FOL Sentence

A **sentence** in FOL is a formula that is **either True or False** under an interpretation.

Examples:

- Birthday(John, 15-Aug) (True or False depending on John)
- ∀x (dog(x) ⇒ mammal(x)) (True or False in the world)
- x ∨ Birthday(x,y) (not a sentence - they have **free variables**)

Rule: A FOL sentence must have **no free variables** - all variables must be bound by a quantifier. A variable is **free** if it is **not inside the scope of a quantifier** such as ∀ **or** ∃**.**

Semantics (Meaning)

Goal: How truth is determined.

- In **Propositional Logic**: truth depends only on assigned truth values.
- In **FOL**: truth depends on
  - the **domain D** (the set of all objects) and
  - the **interpretation** (what each predicate means for each object)

Example:

- Domain D = {x1, x2}
- P(x1) = True, P(x2) = False

If x = x1:

- P(x) is True
- ¬P(x) is False

If x = x2:

- P(x) is False
- ¬P(x) is True

Which values make ∀x P(x) (For **all** x in the domain, P(x) is true) or ∃x ¬P(x) (There exists **some** x in domain such that P(x) is false) true.?

Activity:

x	P(x)	¬P(x)
x1	T	F
x2	F	T

Nested Quantifiers

- Predicate: $Loves(x,y)$ means “x loves y”.
  Question:
- “What does **∀x ∃y Loves(x,y)** mean?”
- Read inside-out:
  - For **each person x**, there is **some person y** such that **x loves y**.
    This means: Everyone loves *someone*.
- Then if we change order:
- **∃y ∀x Loves(x,y)**
  - There is **one person y** such that **every person x** loves **that same y**.
    This means: There is someone loved by everyone.

- **Key point:** Order of ∀ and ∃ matters - swapping them **changes the meaning completely.**

Nested Quantifiers

English sentence	FOL formula	Meaning explained simply
Someone loves everyone	$\exists x \forall y$ Loves(x,y)	There exists a person x who loves every y
There is someone who does not love anyone	$\exists x \forall y$ $\neg$Loves(x,y)	There is a person x who loves no one
There is someone who is not loved by anyone	$\exists x \forall y$ $\neg$Loves(y,x)	There is a person x who is unloved by everyone
Everyone loves everyone	$\forall x \forall y$ Loves(x,y)	All people love all people
There is someone who doesn't love themself	$\exists x \neg$Loves(x,x)	There is a person who doesn't love themselves

Introducing Equality in FOL

Problem:

- Normally in FOL you cannot say =.
- But **FOL with equality** allows it.

Example: Numbers

- `successor(x), predecessor(x), equal(x,y)`

Option 1 (without =):

$\forall x \quad equal(x, successor(predecessor(x)))$

Option 2 (with =):

$\forall x \quad (successor(predecessor(x)) = x)$

This is clearer and allowed when using **FOL with equality**.

Key point:

- `term = term` is allowed as a sentence
- Functions like `successor(predecessor(x))` produce terms, so they can be compared.

DeMorgan Rules

$$\neg \forall x\, P(x) \equiv \exists x\, \neg P(x)$$
$$\neg \exists x\, P(x) \equiv \forall x\, \neg P(x)$$

DeMorgan Rules

Recall De Morgan's Laws (Propositional Logic)

Start with something familiar:

- $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
- $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$

We can prove them using a truth table.

De Morgan's Laws in FOL (Inside a Formula)

Example:

$$\neg(even(x) \vee div4(x)) \equiv \neg even(x) \wedge \neg div4(x)$$

This shows:

- Logical connectives inside FOL work just like propositional logic.

Equivalence in FOL

Recall: What "Equivalence" Means

Definition:

Two formulas $\mathbf{F_1}$ and $\mathbf{F_2}$ are **equivalent**

if and only if $\mathbf{F_1} \Leftrightarrow \mathbf{F_2}$ **is valid** (true under **all** interpretations).

Example (Propositional):

- $\neg(P \land Q) \equiv \neg P \lor \neg Q$

A truth table can be used to prove this in propositional logic.

Step 2 - Why Truth Tables Don't Work for FOL

Explain:

- FOL has **quantifiers** ($\forall$, $\exists$)
- Quantifiers range over **possibly infinite domains** (like all people, all numbers)
- Truth tables only work with **finite sets of propositional variables**

So in FOL, you **cannot list all cases** to check truth.

Instead:

We rely on **known equivalence laws** and **logical transformations**.

Equivalence in FOL

Known Equivalences to Use:

- Implication:
  - $P \Rightarrow Q \equiv \neg P \vee Q$

- De Morgan's laws for quantifiers:
  - $\neg \forall x\, P(x) \equiv \exists x \neg P(x)$
  - $\neg \exists x\, P(x) \equiv \forall x \neg P(x)$

- Distributing quantifiers (with care):
  - If x not free in Q:

    $\forall x\, (P(x) \wedge Q) \equiv (\forall x P(x)) \wedge Q$
  - "If the quantifier doesn't care about a part of the formula, we can safely move that part outside

Equivalence in FOL

Category	Equivalence (Symbolic)	Plain English Explanation
Double Negation	$\neg(\neg P) \equiv P$	"Not not P" is the same as P
De Morgan (Propositional)	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$	Negation flips AND to OR, OR to AND
Implication	$P \rightarrow Q \equiv \neg P \vee Q$	"If P then Q" is equivalent to "not P or Q"
Biconditional	$P \Leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$	"P if and only if Q" means both directions hold
Negation of Quantifiers	$\neg\forall x\, P(x) \equiv \exists x\, \neg P(x)$ $\neg\exists x\, P(x) \equiv \forall x\, \neg P(x)$	"Not all" ≡ "There exists a counterexample"; "Not exists" ≡ "All fail"

Equivalence in FOL

Category	Equivalence (Symbolic)	Plain English Explanation
Distribution of ∧ / ∨	$\forall x\,(P(x) \wedge Q(x)) \equiv (\forall x\,P(x)) \wedge (\forall x\,Q(x))$ $\exists x\,(P(x) \vee Q(x)) \equiv (\exists x\,P(x)) \vee (\exists x\,Q(x))$	Quantifiers distribute over AND/OR (when safe)
Pulling out independent terms	If x not free in Q: $\forall x\,(P(x) \wedge Q) \equiv (\forall x\,P(x)) \wedge Q$ $\exists x\,(P(x) \vee Q) \equiv (\exists x\,P(x)) \vee Q$	If Q does not depend on x, it can be pulled out of the quantifier
Commuting Quantifiers	$\forall x\,\forall y\,P(x,y) \equiv \forall y\,\forall x\,P(x,y)$ $\exists x\,\exists y\,P(x,y) \equiv \exists y\,\exists x\,P(x,y)$	Same quantifiers can swap order; different ones usually can't
Equality / Identity	$\forall x\,(x = x)$ (reflexive) $\forall x \forall y\,(x=y \rightarrow y=x)$ (symmetric) $\forall x \forall y \forall z\,((x=y \wedge y=z) \rightarrow x=z)$ (transitive)	Properties of equality

Rules of Inference

"In propositional logic we deal with whole statements, but in FOL we first replace variables with **specific terms** (like a) before applying these rules."

Example:

- General: $\forall x$ (Student(x) $\rightarrow$ Studies(x))
- Instantiated: Student(Alice) $\rightarrow$ Studies(Alice)
- Now apply Modus Ponens if we know Student(Alice).

Rules of Inference

1. Modus Ponens (Direct reasoning):
P(a),
P(a) → Q(a)
– – – – – –
Q(a)

Read as : "If P is true, and P implies Q, then Q must be true.”

2. Modus Tollens (Reasoning by contrapositive):
¬Q(a),
P(a) → Q(a)
– – – – – –
¬P(a)
Read as: "If P implies Q, and Q didn't happen, then P can't be true.”

Rules of Inference

1. Hypothetical Syllogism (Chaining implications):
   $P(a) \rightarrow Q(a)$,
   $Q(a) \rightarrow R(a)$
   – – – – – – – –
   $P(a) \rightarrow R(a)$

Read as: "If P leads to Q, and Q leads to R, then P leads to R."

Rules of Inference

1. Or Introduction :

$$\frac{P(a)}{P(a) \vee Q(a)}$$

“If P is true, then P or Q is also true, no matter what Q is."
Analogy: "If Alice passed the exam, then either Alice passed or Bob passed."

Rules of Inference

2. Or Elimination

$P(a) \vee Q(a)$,
$P(a) \rightarrow R(a)$,
$Q(a) \rightarrow R(a)$

$R(a)$

"If we know P or Q is true, and both P and Q lead to the same conclusion R, then R must be true."
Analogy: "If either it rains or it snows, then in both cases the ground will get wet → so the ground will get wet."

Rules of Inference

3. And Introduction :

P(a),
Q(a)
———————-
P(a) ^ Q(a)

"If we know P is true and Q is true, then both are true together." Analogy:
"Alice is tall. Alice is smart. Therefore, Alice is tall and smart."

Rules of Inference

4. And Elimination:

Symbolic:
P(a) ^ Q(a)
————————-
P(a)

"If P and Q are true together, then P is true by itself (and so is Q)."
Analogy: "Alice is tall and smart → so Alice is tall

Rules of Inference

5. Resolution Rule :

(A(a) v B(a) v ¬C(a)),
(¬A(a) v ¬E(a))
———————-
B(a)∨¬C(a)∨¬E(a)

"If one statement contains A and another contains -A, you can cancel them and combine what's left.”

Rules of Inference with Quantifiers

1. Universal Elimination (Instantiation)

Rule:
$\forall$ xP(x)
———-
P(c)

If something is true for all elements, it must be true for any specific element.

Example:
$\forall$ x Likes(x, flower)
Substituting x = Sagar $\rightarrow$ Likes(Sagar, flower)

Rules of Inference with Quantifiers

2. Existential Elimination (Instantiation / Skolemization)

Rule:
$\exists x\, P(x)$
———-
$P(c)$

If there exists some element with property P, introduce a placeholder constant c to represent "that element."

Example:
$\exists x$ Likes(x, flower)
$\rightarrow$ Likes(c, flower) (where c is a new symbol, not used before)
Caution : c is just a witness - we don't know who exactly it is. Analogy:

Rules of Inference with Quantifiers

3. Existential Introduction (Generalization)
Rule:
P(c)
———————
∃xP(x)

If a property is true for some specific constant, then it's true that there exists such an element.

Example:
Likes (Sagar, flower)
→ ∃x Likes(x, flower)

Substitution & Reduction to Propositional Inference

Substitution Basics

Definition: Substitution means replacing a variable with a constant.

- SUBST({x/49}, Perfect_sq(x)) → Perfect_sq(49)
- SUBST({x/49, y/7}, Divides(x,y)) → Divides(49,7)

Key Point: Substitution grounds a variable → turns a general statement into a specific one.

Substitution & Reduction to Propositional Inference

Rules of Instantiation

Universal Instantiation (UI)

- Rule: From ∀x P(x), we can infer P(c) for any constant c.
- Example: ∀x Likes(x, flower) → Likes(Sagar, flower).
- "If everyone in the class likes pizza, then Sagar likes pizza too."

Important: UI can be applied *many times* with different constants.

$$\forall v, \alpha \quad \Longrightarrow \quad SUBST(v/g, \alpha)$$

means:

- If you have **"for all v, α"** (a universally quantified sentence),
- then you are allowed to **substitute v with any ground term g** (a concrete constant like 49, John, etc.)
- and get a valid instance of α.

This notation is just a **formal definition of Universal Instantiation** using the substitution function.

Substitution & Reduction to Propositional Inference

Existential Instantiation (UI)

- Original existential statement:

$$\exists v \; \alpha(v)$$

- EI introduces a **new constant symbol** k (a "placeholder") that **does not already exist in the KB**.
- Substitution:

$$\text{SUBST}(v/k, \alpha) \quad \text{gives a ground fact we can use in reasoning.}$$

Substitution & Reduction to Propositional Inference

Example Knowledge Base (KB):

1. $\forall x$ (King(x) $\wedge$ Greedy(x) $\rightarrow$ Evil(x))
2. King(John)
3. Greedy(John)
4. Brother(Richard, John)

Query: Evil(John)?

Step 1: Apply UI to $\forall x$ …

- SUBST{x/John}: King(John) $\wedge$ Greedy(John) $\rightarrow$ Evil(John)
- SUBST{x/Richard}: King(Richard) $\wedge$ Greedy(Richard) $\rightarrow$ Evil(Richard)

Step 2: Discard $\forall$ sentence → KB now is propositional:

- King(John)
- Greedy(John)
- Brother(Richard, John)
- King(John) $\wedge$ Greedy(John) $\rightarrow$ Evil(John)

Step 3: Apply Modus Ponens → Evil(John)

Substitution & Reduction to Propositional Inference

Why function symbols → *infinite* ground terms

- **Ground term/sentences** = a term with no variables (e.g., $\mathtt{John}$, 42).
- If your language has a **constant** $\mathtt{John}$ and a **unary function** $\mathtt{Father}(\cdot)$, you can build:
  - $\mathtt{John}$
  - $\mathtt{Father}(\mathtt{John})$
  - $\mathtt{Father}(\mathtt{Father}(\mathtt{John}))$
  - $\mathtt{Father}(\mathtt{Father}(\mathtt{Father}(\mathtt{John})))$
  - …and so on forever.

So the **Herbrand universe** (the set of all ground terms) becomes infinite whenever you have at least one function symbol and at least one constant.

Because quantifier-elimination (propositionalization) needs to instantiate variables with *all* ground terms, you get infinitely many instances i.e., an infinite propositional theory.

Generalized Modus Ponens (GMP)

For atomic sentences p_i, p_i' ,and q, where there is a substitution such that

$SUBST(\theta, p_i') = SUBST(\theta, p_i)$, ∀ i,

$$\frac{p_1',\ \ p_2',\ \ \ldots,\ \ p_n',\ \ (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}\ .$$

Where:

- $P_1, P_2, \ldots, P_n$ → atomic sentences (facts) in your KB.
- $P_1 \wedge P_2 \wedge \cdots \wedge P_n \rightarrow q$ → a rule in the KB (implication).
- θ → substitution (mapping of variables to constants) that makes the premises match the facts.
- q^{θ} → the conclusion after applying the substitution.

Unification

Unification finds substitutions that make two expressions **look identical**.
It's like solving a puzzle: "How can I rename variables so both sides match?"

Formal Rule:

$$UNIFY(p, q) = \theta \quad \text{where} \quad SUBST(\theta, p) = SUBST(\theta, q)$$

Examples :

1. UNIFY(Knows(John, x), Knows(John, Jane)) = {x/Jane}
   Replace x with Jane → they match.
2. UNIFY(Knows(John, x), Knows(y, Bill)) = {x/Bill, y/John}
   Replace x with Bill, y with John.
3. UNIFY(Knows(John, x), Knows(y, Mother(y))) = {y/John, x/Mother(John)}
   Replace y with John, x with Mother(John).
4. UNIFY(Knows(John, x), Knows(x, Elizabeth)) = **fail**
   x can't be both "John" and "Elizabeth."

Unification

“Name clashes”

- If variables clash (like x being reused), **standardize apart** by renaming.
- Example: Knows(John, x) and Knows(x, Elizabeth) → rename → Knows(John, x1), Knows(x2, Elizabeth).
- Now UNIFY = {x1/Elizabeth, x2/John}.

We want to correctly formalize the English statement: *"All Germans speak the same language."* Using the predicate `speaks(x, l)` meaning "person x speaks language l", and `German(x)` meaning "x is German":

A) $\forall x(German(x) \rightarrow \exists l\ speaks(x, l))$

B) $\exists l\ \forall x(German(x) \rightarrow speaks(x, l))$

C) $\forall x, \forall y, (German(x) \land German(y)) \rightarrow \forall l, (speaks(x, l) \leftrightarrow speaks(y, l)))$

D) $\forall x, (German(x) \rightarrow \forall y(German(y) \rightarrow \exists l, (speaks(x, l) \land speaks(y, l))))$

Find the Most General Unifier (MGU) for the following sets of formulas:

$$p(a, x, f(g(x))) \quad \text{and} \quad p(z, h(z, w), f(w))$$

Which of the following substitutions is correct?

A) $\{z/a, h(a, w)/x, g(x)/w\}$

B) $\{z/a, x = h(a, w), w = g(x)\}$

C) $\{z = a, x = h(a, g(x))\}$

D) None of the above

9.4 For each pair of atomic sentences, give the most general unifier if it exists:

a. $P(A, B, B)$, $P(x, y, z)$.

b. $Q(y, G(A, B))$, $Q(G(x, x), y)$.

c. $Older(Father(y), y)$, $Older(Father(x), John)$.

d. $Knows(Father(y), y)$, $Knows(x, x)$.

Sentence: The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American. An enemy of America counts as "hostile".

Horn clauses

A **Horn clause** is a **disjunction of literals** (statements that are either true or false) with **at most one positive literal**.

Types of Horn Clauses
There are mainly three forms:

1. **Fact**
   - A clause with **exactly one positive literal and no negative literals**.
   - Example: P
   - Interpretation: “P is true.”

Rule (Implication)

- A clause with **one positive literal and one or more negative literals**.
- Written as an implication: $P \leftarrow Q1 \land Q2 \land \ldots \land Qn$
- This is equivalent to: $\neg Q1 \lor \neg Q2 \lor \ldots \lor \neg Qn \lor P$
- Interpretation: “P is true if all Qi are true.”

Horn clauses

Horn clause	Equivalent rule/fact
$\neg A \vee \neg B \vee C$	$A \wedge B \Rightarrow C$
$\neg A \vee C$	$A \Rightarrow C$
A	Fact A
$\neg A \vee \neg B$	$A \wedge B \Rightarrow False$

DPLL

DPLL (Davis–Putnam–Logemann–Loveland) is an algorithm for deciding whether a **propositional logic formula in CNF is satisfiable**.

Satisfiability:

Given a propositional sentence, does there exist a truth assignment to its propositions that makes the entire sentence **True***?*

In simple words:

DPLL tries different truth assignments, but uses logical shortcuts to avoid trying every possibility.

Example:

$$(P \vee Q) \wedge (\neg P \vee R)$$

DPLL asks:

Is there a truth assignment to P, Q, R that makes all clauses true?

DPLL

Original formula:

$$(P \vee Q) \wedge (\neg P \vee Q)$$

Set:

$$P = \text{True}$$

First clause:

$$(\text{True} \vee Q) = \text{True}$$

Second clause:

$$(\neg \text{True} \vee Q)$$

$$(\mathit{False} \vee Q)$$

Therefore:

$$Q = \text{True}$$

So we obtain: P=True, Q=True All clauses are satisfied. Therefore: SAT DPLL can stop immediately.

DPLL

With n propositional variables, brute-force search may require:

$$\boxed{2^n}$$

possible truth assignments.

DPLL reduces this search using:

- **Early detection of contradictions**
- **Pure literal elimination**
- **Unit propagation**
- **Branching**

Early detection

Consider the CNF formula:

$$(A \vee B) \wedge (\neg A \vee C)$$

Suppose DPLL assigns:

$$A = \text{True}$$

Then:

$$A \vee B = \text{True} \qquad A \vee C = \text{True}$$

Hence the whole formula is **SAT**.

No need to assign C **or** B.

DPLL can stop as soon as the formula is known to be true.

Pure Literal

Pure symbol: A symbol that appears with only one polarity.

Consider:

$$(A \vee B) \wedge (\neg B \vee C) \wedge (C \vee A)$$

Here, A appears only positively.

$$\boxed{A \text{ is pure}}$$

Therefore, safely assign:

$$A = \text{True}$$

This makes:

$$A \vee B = \text{True}, \qquad C \vee A = \text{True}.$$

A pure symbol can be assigned to make its literal true.

Unit Clause

Unit clause: A clause containing only one remaining unassigned literal.

Consider:

$$(A \vee B) \wedge (\neg B \vee C) \wedge (\neg C)$$

The clause

$$(\neg C)$$

is a unit clause.
Therefore:

$$C = \text{False}$$

Then:

$$(\neg B \vee C) \quad \Rightarrow \quad (\neg B \vee \text{False}) \quad \Rightarrow \quad B = \text{False}$$

Now:

$$(A \vee B) \quad \Rightarrow \quad A = \text{True}$$

Branching

What if there is:

- no contradiction,
- no pure symbol, and
- no unit clause?

Then DPLL must **branch**.

For example:

$$(A \vee B) \wedge (\neg A \vee C)$$

No forced assignment is available.
Choose a symbol, say A:

$$\boxed{A = \text{True}} \quad \text{or} \quad \boxed{A = \text{False}}$$

and recursively explore both possibilities.

Branching is used only when logical deductions cannot decide the value.

Forward Chaining

Key Idea:

- Start with facts.
- Apply rules to infer new facts.
- Repeat until you answer the query (or no new facts appear).

It's a **data-driven approach** → "What can I conclude from what I already know?"

Definite Clauses

• Form: A1 ∧ A2 ∧… ∧ An → B (modus ponen)

Example:

Rule: King(x) ∧ Greedy(x) → Evil(x)

• Facts: King(John), Greedy (John)

• Conclusion: Evil(John)

function PL-FC-ENTAILS?(KB, q) **returns** *true* or *false*

inputs: KB, the knowledge base, a set of propositional definite clauses
q, the query, a proposition symbol

$count \leftarrow$ a table, where $count[c]$ is the number of symbols in c's premise

$inferred \leftarrow$ a table, where $inferred[s]$ is initially *false* for all symbols

$agenda \leftarrow$ a queue of symbols, initially symbols known to be true in KB

while $agenda$ is not empty **do**
  $p \leftarrow$ POP($agenda$)
  if $p = q$ **then return** *true*
  if $inferred[p] = false$ **then**
    $inferred[p] \leftarrow true$
    for each clause c in KB where p is in c.PREMISE **do**
      decrement $count[c]$
      if $count[c] = 0$ **then** add c.CONCLUSION to $agenda$
return *false*

Forward Chaining in PL - Graphical Solution

KB:

Facts: A,B

Rules

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

Query Q

Forward Chaining in PL - Graphical Solution

KB:

A,B

Query Q

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

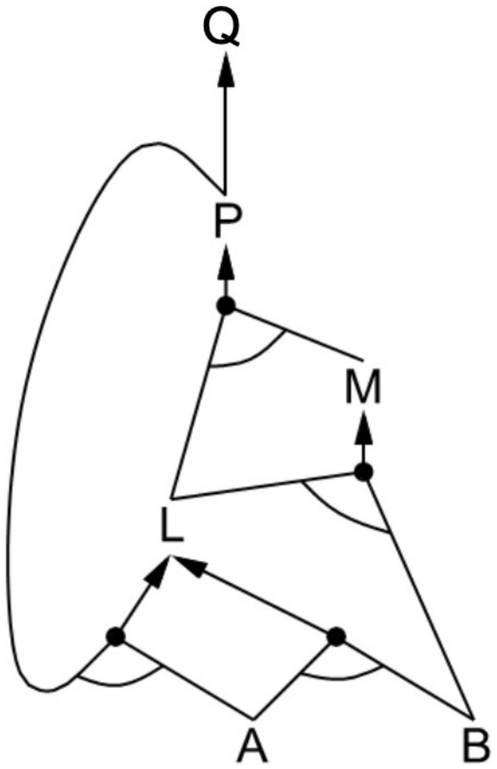
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

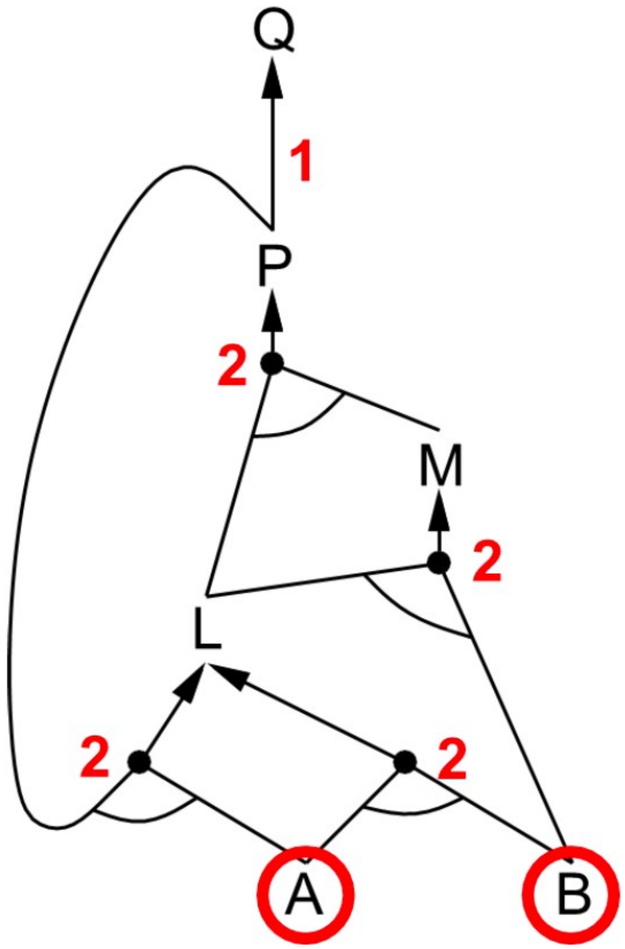
$$A$$

$$B$$



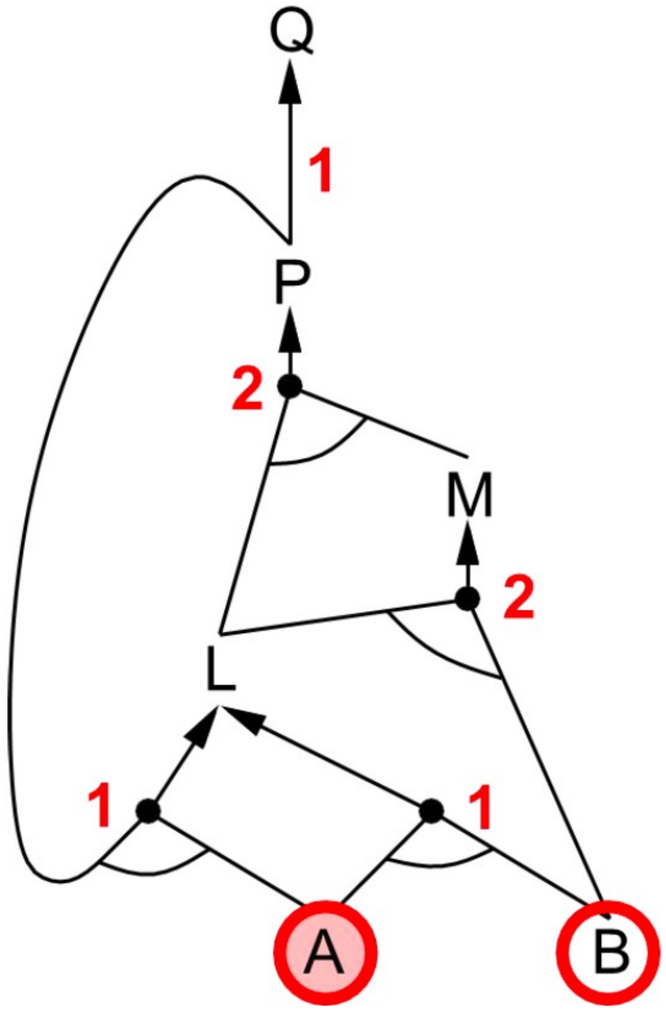
Forward Chaining in PL - Graphical Solution

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$



Forward Chaining in PL - Graphical Solution

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$



Forward Chaining in PL - Graphical Solution

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

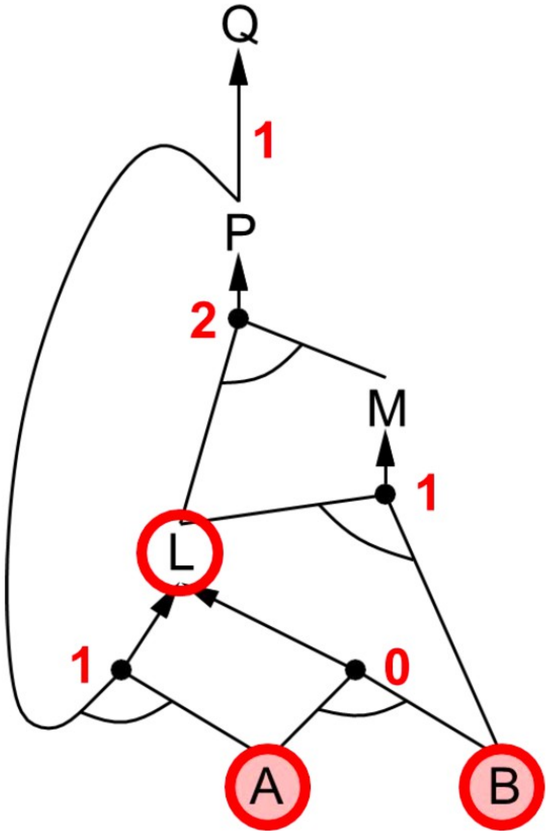
$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B



Forward Chaining in PL - Graphical Solution

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

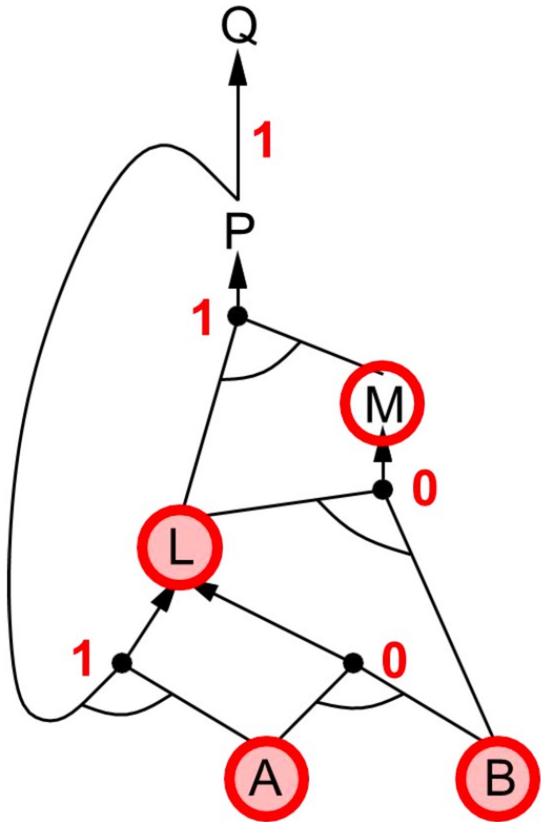
$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

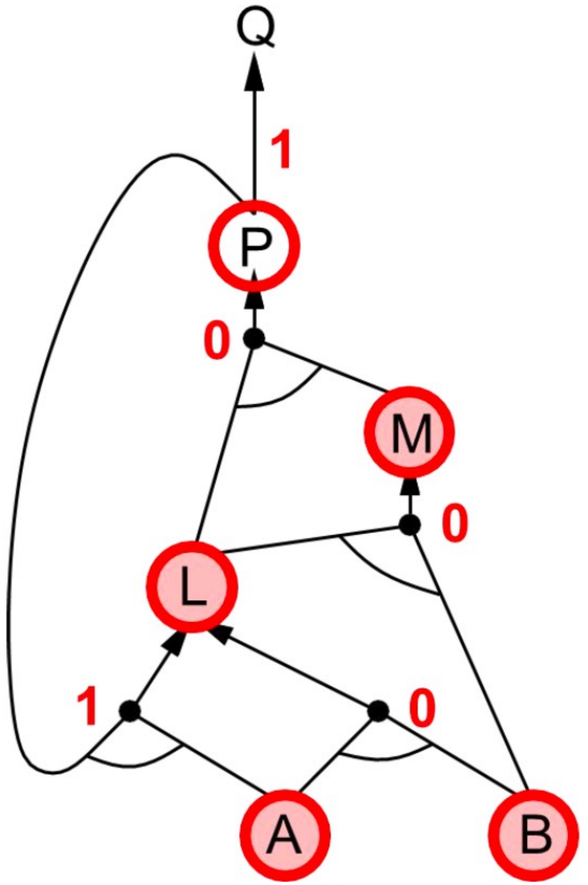
A

B



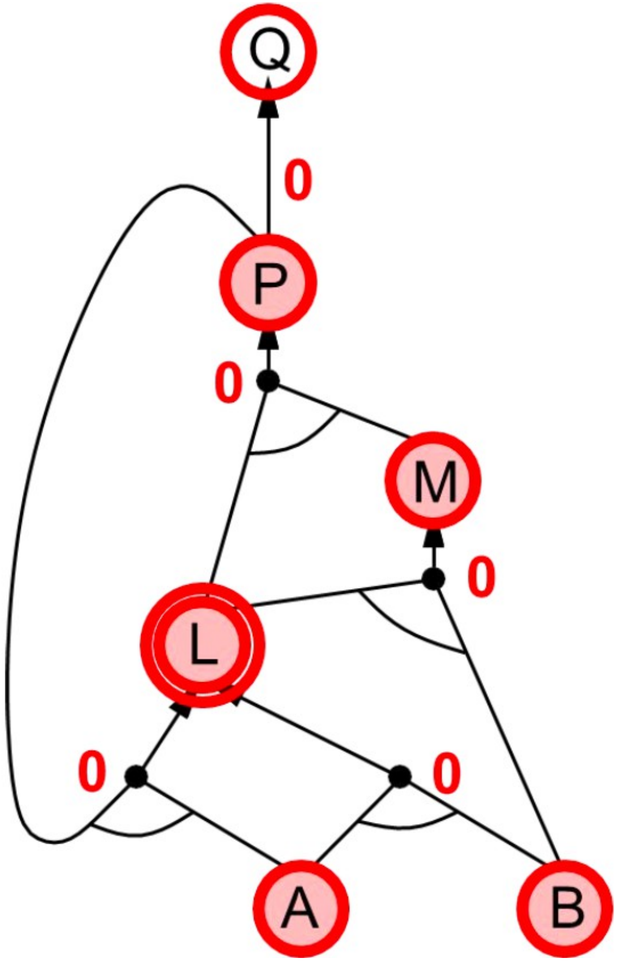
Forward Chaining in PL - Graphical Solution

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$



Forward Chaining in PL - Graphical Solution

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$



Forward Chaining in PL - Graphical Solution

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

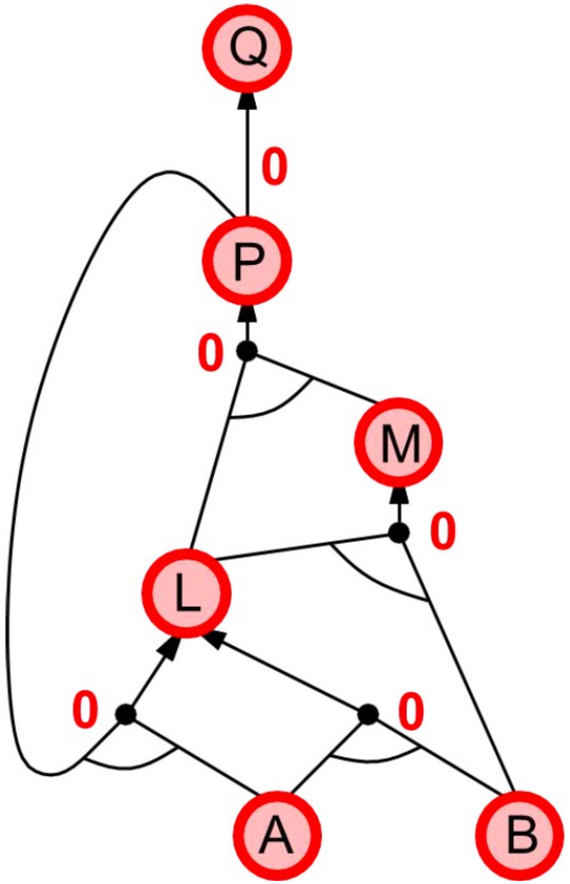
$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B



Forward Chaining - FOL

function FOL-FC-ASK(KB, α) **returns** a substitution or *false*
  inputs: KB, the knowledge base, a set of first-order definite clauses
    α, the query, an atomic sentence
  local variables: *new*, the new sentences inferred on each iteration

  repeat until *new* is empty
    new $\leftarrow \{\}$
    for each *rule* **in** KB **do**
      $(p_1 \wedge \ldots \wedge p_n \Rightarrow q) \leftarrow$ STANDARDIZE-VARIABLES(*rule*)
      for each θ such that SUBST($\theta, p_1 \wedge \ldots \wedge p_n$) = SUBST($\theta, p'_1 \wedge \ldots \wedge p'_n$)
          for some $p'_1, \ldots, p'_n$ in KB
        $q' \leftarrow$ SUBST(θ, q)
        if q' does not unify with some sentence already in KB or *new* **then**
          add q' to *new*
          $\phi \leftarrow$ UNIFY(q', α)
          if ϕ is not *fail* **then return** ϕ
    add *new* to KB
  return *false*

Forward Chaining

Classic Example (West is a Criminal)

Knowledge Base:

1. $\forall x,y,z$ American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\rightarrow$ Criminal(x)
2. $\exists x$ Missile(x) $\wedge$ Owns(Nono,x)
3. $\forall x$ Missile(x) $\wedge$ Owns(Nono,x) $\rightarrow$ Sells(West,x,Nono)
4. $\forall x$ Missile(x) $\rightarrow$ Weapon(x)
5. $\forall x$ Enemy(x,America) $\rightarrow$ Hostile(x)
6. American(West)
7. Enemy(Nono, America)

Query: Criminal(West)?

Forward Chaining

Step 1: Eliminate ∃ (Existential Instantiation)

From (2): Introduce constant M1:

We **introduce a new constant**, call it M1, to stand for "some missile owned by Nono".

- Missile(M1), Owns(Nono, M1). — Eliminate ∃ (Existential Instantiation)

Step 2: Apply Forward Chaining Rules

- From (4): Missile(M1) → Weapon(M1). — Universal Instantiation
- From (3): Missile(M1) ∧ Owns(Nono, M1) → Sells(West, M1, Nono). UI
- From (5): Enemy(Nono, America) → Hostile(Nono). UI

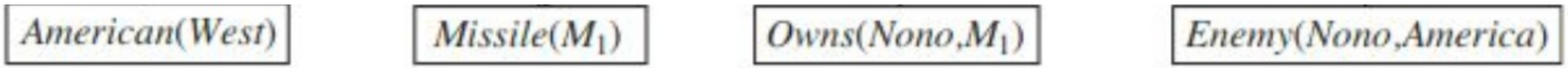
Step 3: Apply Master Rule (1)

American(West) ∧ Weapon(M1) ∧ Sells(West, M1, Nono) ∧ Hostile(Nono) →

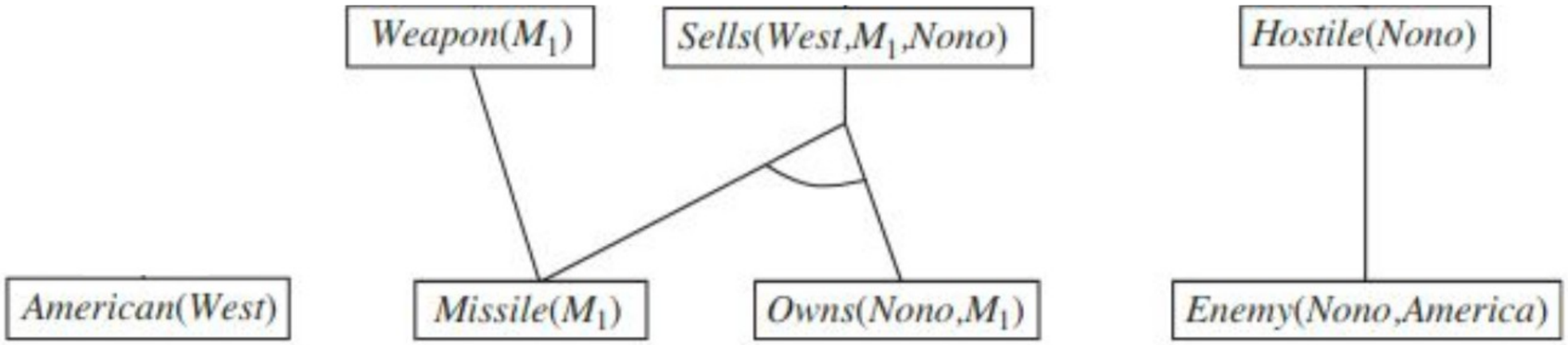
Criminal(West).

Therefore, Criminal(West) is proved.

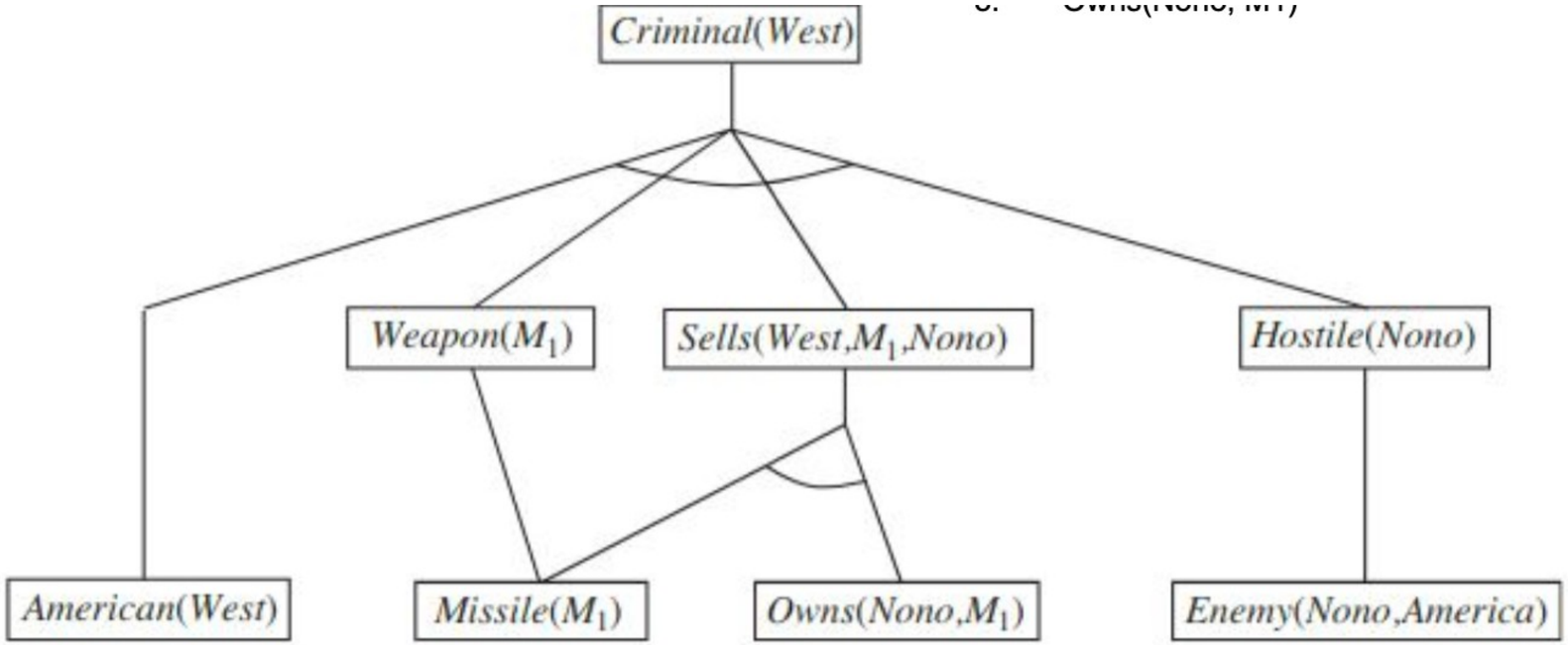
Forward Chaining



Forward Chaining



Forward Chaining



Problem	What happens?	Solution
1. Pattern matching	Finding substitutions can be expensive	Better matching / indexing / conjunct ordering
2. Repeated matching	Same rules are checked again and again	Incremental forward chaining / RETE
3. Irrelevant inference	Derives facts unrelated to the query	Backward chaining / Magic Sets

Forward Chaining

1. What does a query look like in forward chaining?

- In forward chaining, the query is usually **a single literal** like:
  - "Is Q true?"
- Or at most a **conjunction**:
  - "Are Q and R both true?"

Why? Because forward chaining works by **deriving individual facts** step by step using rules of the form:

$$(P_1 \wedge P_2 \wedge \cdots \wedge P_n) \rightarrow Q$$

Forward Chaining

2. What if the query is a disjunction?

Suppose the query is:

$$Q \vee R$$

("Is Q or R true?")

To answer this, the forward chaining engine would have to reason like this:

- If I can prove Q, the query is satisfied.
- If I can prove R, the query is also satisfied.
- But what if neither is derivable? Then I cannot conclude $Q \vee R$, **even though it may still be logically valid**.

Example:

A tautology like $P \vee \neg P$ is always true. But forward chaining cannot derive this because it doesn't have rules for reasoning about disjunction directly — it only fires rules when all antecedents (conjuncts) are True

Forward Chaining

3. Why is this a problem?

- Forward chaining is **syntactic and mechanical**: it adds new facts only when antecedents are fully satisfied.
- It **does not explore alternatives** or perform case splits ("maybe Q, maybe R").
- A disjunctive query requires **non-deterministic branching** (e.g., trying one case, then the other). That's what algorithms like **resolution** or **model checking (SAT)** handle, not forward chaining.
- Sound but not always complete

Backward Chaining

Motivation

- Question: *"How does an AI system like an expert system answer a query?"*

- Ex. *Doctor diagnosing a disease backward from symptoms (goal) to possible causes (facts).*
- **forward chaining** (data → conclusions) vs. **backward chaining** (goal → supporting facts).

- Definition: Backward chaining works backward from the query (goal) and tries to prove it by finding rules whose conclusion matches the goal, then proving their premises recursively.

```
function FOL-BC-ASK(KB, query) returns a generator of substitutions
  return FOL-BC-OR(KB, query, { })

---

generator FOL-BC-OR(KB, goal, θ) yields a substitution
  for each rule (lhs ⇒ rhs) in FETCH-RULES-FOR-GOAL(KB, goal) do
    (lhs, rhs) ← STANDARDIZE-VARIABLES((lhs, rhs))
    for each θ′ in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal, θ)) do
      yield θ′

---

generator FOL-BC-AND(KB, goals, θ) yields a substitution
  if θ = failure then return
  else if LENGTH(goals) = 0 then yield θ
  else do
    first, rest ← FIRST(goals), REST(goals)
    for each θ′ in FOL-BC-OR(KB, SUBST(θ, first), θ) do
      for each θ″ in FOL-BC-AND(KB, rest, θ′) do
        yield θ″
```

Figure 9.6 A simple backward-chaining algorithm for first-order knowledge bases.

Backward Chaining

Knowledge Base (KB)

1. Rain → Wet
2. Sprinkler → Wet
3. Cloudy → Rain
4. Cloudy (fact)

Query: Is Wet true?

Step-by-step Backward Chaining

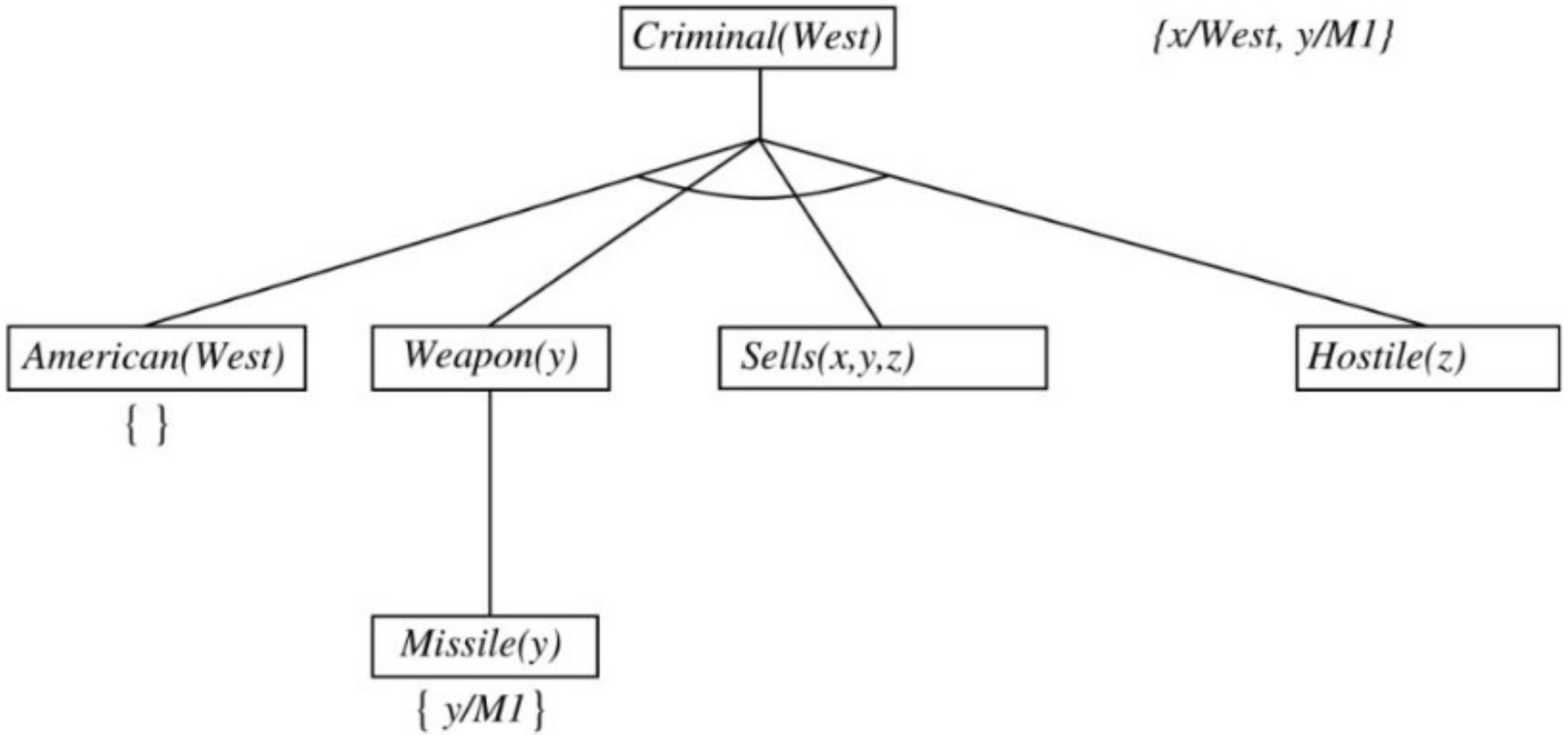
1. **Goal:** Wet
   - Look in KB: Rules (1) and (2) can produce Wet.
   - So possible subgoals: Rain (from 1) or Sprinkler (from 2).
2. **Branch A: Try rule (1)**
   - Subgoal: Rain
   - Look in KB: Rule (3) can produce Rain.
   - So new subgoal: Cloudy.
   - Fact (4): Cloudy is already true
   - So Rain is true → Wet is true (via rule 1).
3. **Branch B: Try rule (2)** (just to check another path)
   - Subgoal: Sprinkler
   - But there's no fact or rule for Sprinkler → fails.

Backward Chaining

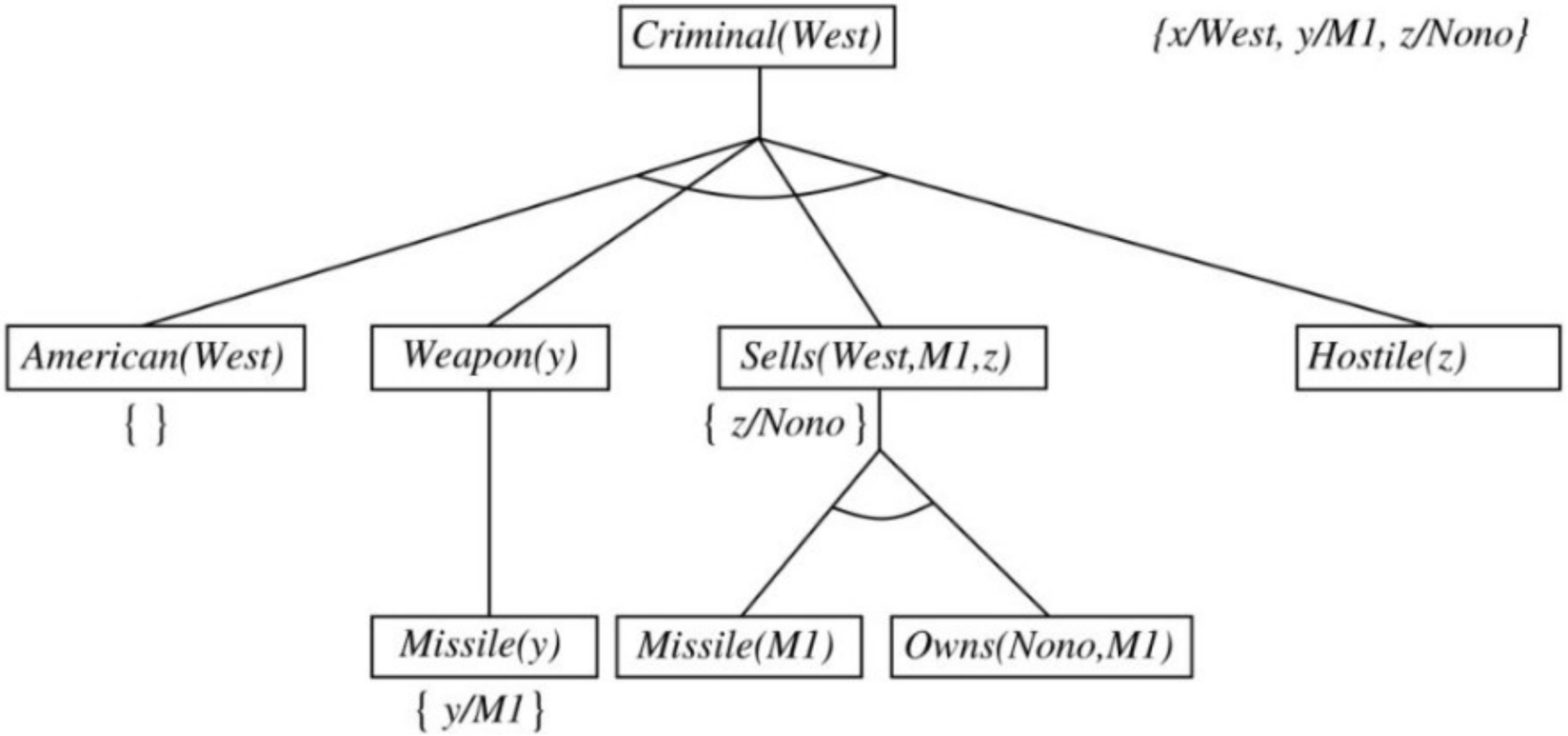
- **Goal:** Criminal(West) → matches rule (1) with substitution {x/West} → create subgoals:
  - American(West), Weapon(y), Sells(West,y,z), Hostile(z).
- American(West) matches fact (5) → success.
- Weapon(y) → use (3): Missile(y) → Weapon(y) → need Missile(y).
  - Missile(M1) is a fact (7) → unify {y/M1} → success for Weapon(M1).
- Sells(West,y,z) with {y/M1} becomes Sells(West,M1,z) → rule (2) gives Sells(West,x,Nono) when x=M1 with {x/M1,z/Nono} → antecedents Missile(M1) and Owns(Nono,M1) are facts (7) & (8) → success.
- Hostile(z) → use (4): Enemy(z,America) → Hostile(z) →

  Enemy(Nono,America) is a fact (6) → unify {z/Nono} → success.
- All subgoals succeed under the composed substitution {x/West,y/M1,z/Nono} → Criminal(West) proven.

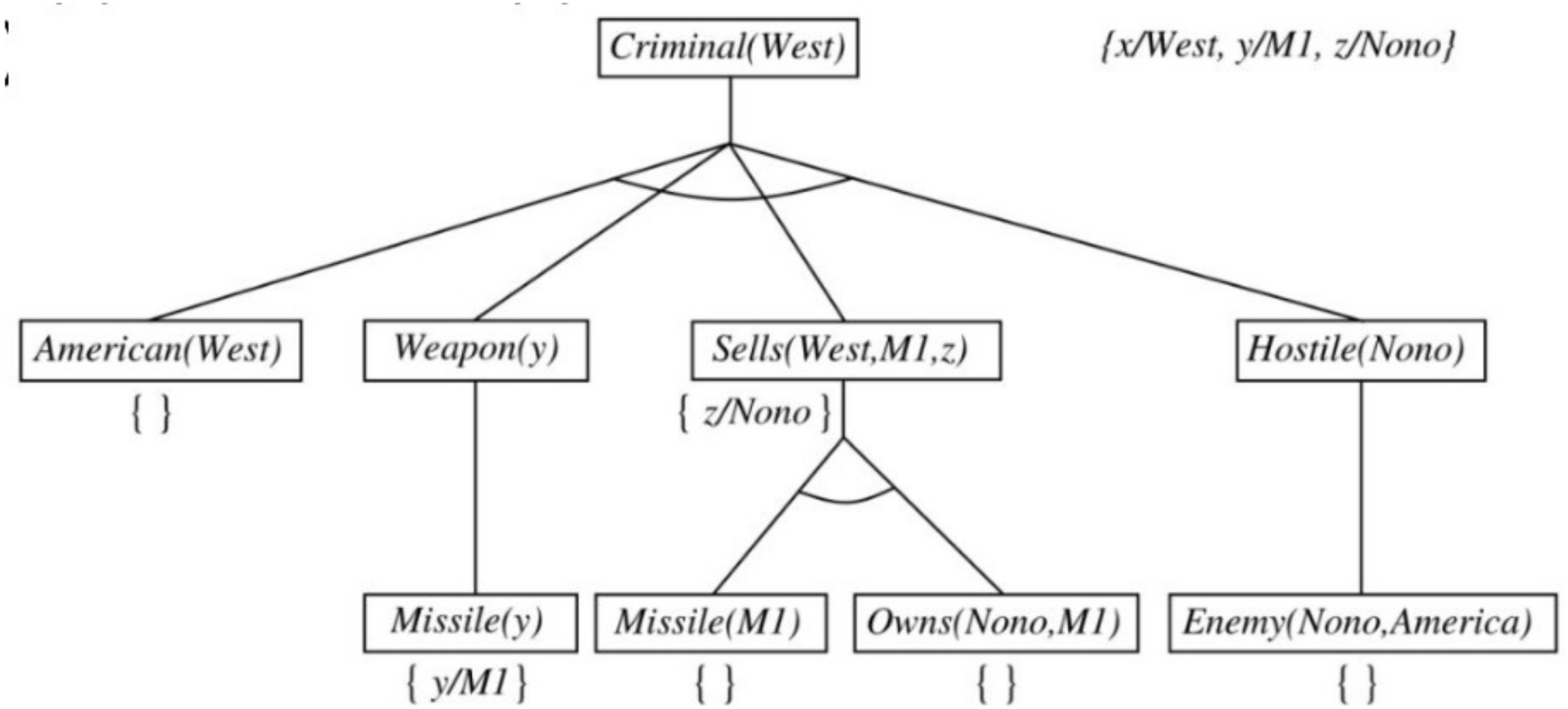
Backward Chaining - Graphical Solution



Backward Chaining - Graphical Solution



Backward Chaining - Graphical Solution



Question

1. $\forall x \ \ Student(x) \ \wedge Studies(x) \ \rightarrow Pass(x)$
2. $\forall x \ \ LikesIceCream(x) \ \rightarrow Happy(x)$
3. $Student(Alice)$
4. $Studies(Alice)$
5. $LikesIceCream(Alice)$

Query: $Pass(Alice)$ and $Happy(Alice)$

Forward Chaining

Step-by-step:

1. **Facts:** $Student(Alice)$, $Studies(Alice)$, $LikesIceCream(Alice)$
2. **Rule 1:** $Student(x) \land Studies(x) \rightarrow Pass(x)$
   - Substitute $x = Alice \rightarrow$ antecedent satisfied ($Student(Alice)$ and $Studies(Alice)$ true) $\rightarrow$ infer $Pass(Alice)$
   - **Rule 2:** $LikesIceCream(x) \rightarrow Happy(x)$
   - Substitute $x = Alice \rightarrow$ antecedent satisfied $\rightarrow$ infer $Happy(Alice)$

Query reached: $Pass(Alice)$ and $Happy(Alice)$

Backward Chaining

Idea: Start from **query** and work backwards to see if we can prove it using facts.

Query: $\mathrm{Pass(Alice)}$

1. Look for rules whose consequent matches query → Rule 1: $\mathrm{Student(x)} \wedge \mathrm{Studies(x)} \rightarrow \mathrm{Pass(x)}$
2. Subgoal: $\mathrm{Student(Alice)}$ → already a fact
3. Subgoal: $\mathrm{Studies(Alice)}$ → already a fact
4. Both subgoals satisfied → $\mathrm{Pass(Alice)}$ proven

Query: $\mathrm{Happy(Alice)}$

1. Look for rules whose consequent matches query → Rule 2: $\mathrm{LikesIceCream(x)} \rightarrow \mathrm{Happy(x)}$
2. Subgoal: $\mathrm{LikesIceCream(Alice)}$ → fact
3. Subgoal satisfied → $\mathrm{Happy(Alice)}$ proven

Question

Knowledge Base (KB) in Natural Language:

1. Every person who studies hard and attends lectures passes the exam.
2. Anyone who passes the exam and submits the project gets a certificate.
3. All students who enjoy learning like solving puzzles.
4. Alice is a student who studies hard and attends lectures.
5. Alice submits the project.
6. Alice enjoys learning.

Query: Does Alice get a certificate?

Backward Chaining

Solution Steps (S)

Step 1: Convert sentences into FOL

1. $\forall x \ \text{Student}(x) \land \text{StudiesHard}(x) \land \text{AttendsLectures}(x) \rightarrow \text{Pass}(x)$
2. $\forall x \ \text{Pass}(x) \land \text{SubmitsProject}(x) \rightarrow \text{Certificate}(x)$
3. $\forall x \ \text{Student}(x) \land \text{EnjoysLearning}(x) \rightarrow \text{LikesPuzzles}(x)$
4. $\text{Student}(\text{Alice}) \land \text{StudiesHard}(\text{Alice}) \land \text{AttendsLectures}(\text{Alice})$ (fact)
5. $\text{SubmitsProject}(\text{Alice})$ (fact)
6. $\text{EnjoysLearning}(\text{Alice})$ (fact)

Query: $\text{Certificate}(\text{Alice})$

Solution

Step 2: Forward Chaining

1. **Facts:** $Student(Alice), StudiesHard(Alice), AttendsLectures(Alice)$
2. **Rule 1:** $Student \land StudiesHard \land AttendsLectures \rightarrow Pass$
   - Antecedent satisfied → infer $Pass(Alice)$
3. **Rule 2:** $Pass \land SubmitsProject \rightarrow Certificate$
   - Antecedent satisfied ($Pass(Alice)$ just inferred + $SubmitsProject(Alice)$ fact) → infer $Certificate(Alice)$

Query reached: $Certificate(Alice)$

Step 3: Backward Chaining

1. **Goal:** $Certificate(Alice)$
2. Look for rules with consequent $Certificate(x)$ → Rule 2: $Pass(x) \land SubmitsProject(x)$
3. Subgoals:
   - $Pass(Alice)$ → need Rule 1: $Student \land StudiesHard \land AttendsLectures$
     - All satisfied by facts → $Pass(Alice)$
   - $SubmitsProject(Alice)$ → fact
4. Both subgoals satisfied → $Certificate(Alice)$ proven

Step 4: Optional Subgoal (extra info)

- From Rule 3: $Student \land EnjoysLearning \rightarrow LikesPuzzles$
  - Antecedent satisfied → infer $LikesPuzzles(Alice)$

Disadvantages of Backward Chaining

- **Repeated states:** The same subgoal may be generated repeatedly, causing redundant work or loops.
- **Infinite loops:** Depth-first search can follow an infinite branch and never return.
- **Incompleteness:** It may fail to find an existing proof if it gets stuck in an infinite branch before exploring other alternatives.
- **Repeated subgoals:** The same subgoal may be proved multiple times through different branches.

Forward Chaining

Classic Example (West is a Criminal)
Knowledge Base:

1. $\forall x,y,z$ American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\rightarrow$ Criminal(x)
2. $\exists x$ Missile(x) $\wedge$ Owns(Nono,x)
3. $\forall x$ Missile(x) $\wedge$ Owns(Nono,x) $\rightarrow$ Sells(West,x,Nono)
4. $\forall x$ Missile(x) $\rightarrow$ Weapon(x)
5. $\forall x$ Enemy(x,America) $\rightarrow$ Hostile(x)
6. American(West)
7. Enemy(Nono, America)

Query: Criminal(West)?

Resolution

- In propositional logic, Resolution Refutation was used to perform inference through proof by contradiction.
- Similar to propositional logic, first-order logic requires the sentences to be in clausal form, i.e., as a disjunction of literals.
- Literals can contain variables, which are assumed to be universally quantified in this case.

  $\forall$ x,y,z (American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x, y, z) $\wedge$ Hostile(z) $\rightarrow$ Criminal (x))

  becomes

  $\forall$ x,y,z ($\neg$American(x) $\vee$ $\neg$Weapon(y) $\vee$ $\neg$Sells(x, y, z) $\vee$ $\neg$Hostile(z) v Criminal (x))
- The process to convert a sentence to clausal form is similar to propositional logic.
- The difference being the need to handle the existential quantifier and variable conflict.

Resolution

Sentence: “Everyone who loves all animals is loved by someone.”

Step 1: Represent in FOL (before quantifiers)

- *“Everyone who loves all animals”* → for every person `x`, if `x` loves all animals `y`, then …
- *“… is loved by someone”* → there exists some person `z` such that `z` loves `x`.

So:

$$\forall x \Big((\forall y \; Animal(y) \Rightarrow Loves(x, y)) \Rightarrow (\exists z \; Loves(z, x)) \Big)$$

Step 2: Eliminate implication (→)

- Recall: $P \Rightarrow Q \equiv \neg P \vee Q$.

$$\forall x \Big(\neg (\forall y \; Animal(y) \Rightarrow Loves(x, y)) \; \vee \; (\exists z \; Loves(z, x)) \Big)$$

Resolution

Sentence: “Everyone who loves all animals is loved by someone.”

Step 3: Push negation in

- Inside: $\neg(\forall y(Animal(y) \Rightarrow Loves(x,y))) \equiv \exists y \neg(Animal(y) \Rightarrow Loves(x,y))$.
- And: $\neg(Animal(y) \Rightarrow Loves(x,y)) \equiv Animal(y) \wedge \neg Loves(x,y)$.

So:

$$\forall x \Big((\exists y(Animal(y) \wedge \neg Loves(x,y))) \ \vee \ (\exists z \ Loves(z,x)) \Big)$$

Resolution

Sentence: “Everyone who loves all animals is loved by someone.”

Step 4: Skolemization (eliminate existentials)

- Bring into Prenex Normal Form first:

$$\forall x \; \exists y \; \exists z \Big((Animal(y) \land \neg Loves(x, y)) \;\lor\; Loves(z, x) \Big)$$

- Now apply Skolemization:
  - For each `x`, there exists a `y = f(x)` and a `z = g(x)`.

    So:

$$\forall x \; \Big((Animal(f(x)) \land \neg Loves(x, f(x))) \;\lor\; Loves(g(x), x) \Big)$$

Prenex Normal Form (PNF) is a standard way of writing first-order logic (FOL) formulas where all the quantifiers are moved to the front of the formula, before any logical connectives.

Resolution

Sentence: “Everyone who loves all animals is loved by someone.”

What is Skolemization?

- **Definition:** Skolemization is the process of **eliminating existential quantifiers (∃)** by introducing **Skolem functions or constants**.
- It ensures that the meaning of the formula is preserved for resolution, but without existential quantifiers.

Why do we need it?

- Resolution refutation requires formulas in **clausal form (CNF)**, where only **universal quantifiers (∀)** remain.
- Existential quantifiers (∃) complicate matters because they introduce "some unknown object exists."
- Skolemization replaces that unknown with a new symbol (constant or function) so the logic engine can handle it deterministically.

Resolution

Imagine this scenario:

You have a rule like:

> "For every person, there exists a locker that belongs to them."

In logic, that's:

$$\forall \text{person } x, \exists \text{locker } y : owns(x, y)$$

Problem

- Computers don't like statements that say **"there exists something"** because they don't know **which one** it is.
- They work better with rules that just say **"for all things, something happens"**. Think of it like:
- "I know each person has a locker"
- But the computer wants a **concrete way to refer to that locker**, not just "some locker exists."

Resolution

Skolemization is the trick

- We **replace the vague "there exists a locker" with a function** that picks a locker for each person.

$$\forall x, owns(x, f(x))$$

- Here, $f(x)$ is like a **locker-assignment function**: given a person x, it tells you exactly which locker belongs to them.
- Now the computer can handle it easily because it's **explicit**.

Resolution

There are two cases:

1. **If the existential quantifier is not inside any universal scope:**

$$\exists x P(x)$$

Replace with a **Skolem constant** `c` :

$$P(c)$$

Meaning: there exists some specific object `c` that makes `P` true.

Resolution

Case 2: Existential within scope of universals

$$\forall x \exists y \, P(x, y)$$

- `y` depends on `x`. So we introduce a **Skolem function** `f(x)`:

$$\forall x \, P(x, f(x))$$

- Interpretation: "For each x, there exists y (depending on x) such that P(x,y) is true."
- The function `f` encodes that dependency.

> Intuition: Skolem functions are like little "machines" that, given values of universal variables, produce the existential variable that makes the formula true.

"For each x, this function returns a y that satisfies the formula."

Resolution

Sentence: “Everyone who loves all animals is loved by someone.”

Step 5: Distribute ∨ over ∧ (CNF form)

Expression:

$$(Animal(f(x)) \land \neg Loves(x, f(x))) \lor Loves(g(x), x)$$

Distribute:

$$(Animal(f(x)) \lor Loves(g(x), x)) \ \land \ (\neg Loves(x, f(x)) \lor Loves(g(x), x))$$

Resolution

Sentence: “Everyone who loves all animals is loved by someone.”

Step 6: Drop universal quantifier (all variables are implicitly ∀ in clausal form)

So final clauses:

1. `Animal(f(x)) ∨ Loves(g(x),x)`
2. `¬Loves(x,f(x)) ∨ Loves(g(x),x)`

Resolution Inference in First-Order Logic (FOL)

- The resolution rule for first-order clauses is simply a lifted version of the propositional resolution rule.
- **"Lifted version of propositional resolution"**
  - In **propositional logic**, resolution works on statements like $P \vee Q$ and $\neg Q \vee R$ and combines them to get $P \vee R$.
  - **First-order logic (FOL)** is like propositional logic but with **variables and functions** (e.g., Loves(John, x)).
  - So, the FOL resolution rule is basically the **same idea**, but it can handle variables using **unification**.

Resolution Inference in First-Order Logic (FOL)

- Two clauses, which are assumed to be standardized apart so that they share no variables, can be resolved if they contain complementary literals.
- **"Standardized apart"**
  - Before resolving two clauses, we make sure **their variables don't accidentally overlap**.
  - Example:
    - Clause 1: Loves(x, y)
    - Clause 2: ¬Loves(x, z)
  - Here, x in the first clause is **different** from x in the second clause, to avoid confusion.
  - This is called **standardizing apart**.

Resolution Inference in First-Order Logic (FOL)

- Propositional literals are complementary if one is the negation of the other.
- **“Complementary literals”**
  - **Propositional literals:** simple statements like P or ¬P.
    - Complementary means **one is the negation of the other**.
    - Example: P and ¬P → complementary.
  - **First-order literals:** like Loves(John, x) and ¬Loves(y, z)
  - Complementary if you can make them **exactly match by substituting variables**.
  - This is called **unification**.
  - Example: Loves(John, x) and ¬Loves(y, Mary) unify with {y/John, x/Mary}.

Resolution Inference in First-Order Logic (FOL)

- We have two **clauses** (groups of statements connected by OR ∨):
  - Clause 1: l 1 ∨ l 2 ∨ . . . ∨ l k
  - Clause 2: m1 ∨ m2 ∨ . . . ∨ mn
- **Goal:** Combine them by **resolving complementary literals** (one literal is the opposite of the other).

$$\frac{l_1 \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_n}{\text{SUBST}(\theta, l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)}$$

θ = UNIFY(li, ¬mj)

This is the **resolvent** in first-order logic.

- Find **complementary literals** (l i and ¬mj)
- Find **unifier θ**
- Remove the resolved literals, apply θ, combine remaining literals.

Take all literals except the ones that canceled, and apply the substitutions from unification."

Resolution Inference in First-Order Logic (FOL)

Example:

1. [Animal(F(x)) ∨ Loves(G(x), x)]
2. [¬Loves(u, v) ∨ ¬Kills(u, v)]

Resolvent: [Animal(F(x)) ∨ ¬Kills(G(x), x)] with θ = {u/G(x), v/x}

Resolution Inference in First-Order Logic (FOL)

Step 1: Find complementary literals

- Look for a literal in one clause that is the **opposite** of a literal in the other.
- Clause 1 has $Loves(G(x), x)$
- Clause 2 has $\neg Loves(u, v)$
- These are **complementary** because one is the negation of the other ($Loves$ vs $\neg Loves$).

Step 2: Find the unifier (θ)

- We need to make the literals **exactly the same** by replacing variables.
- $Loves(G(x), x)$ vs $Loves(u, v)$ → what substitutions make them equal?
  u=G(x),v=x
- This is the **unifier** $\theta = \{u/G(x), v/x\}$

Resolution Inference in First-Order Logic (FOL)

Step 3: Remove the complementary literals

- Remove Loves(G(x), x) from Clause 1
- Remove ¬Loves(u, v) from Clause 2 Remaining literals:
- Clause 1: Animal(F(x))
- Clause 2: ¬Kills(u, v) → apply θ → ¬Kills(G(x), x)

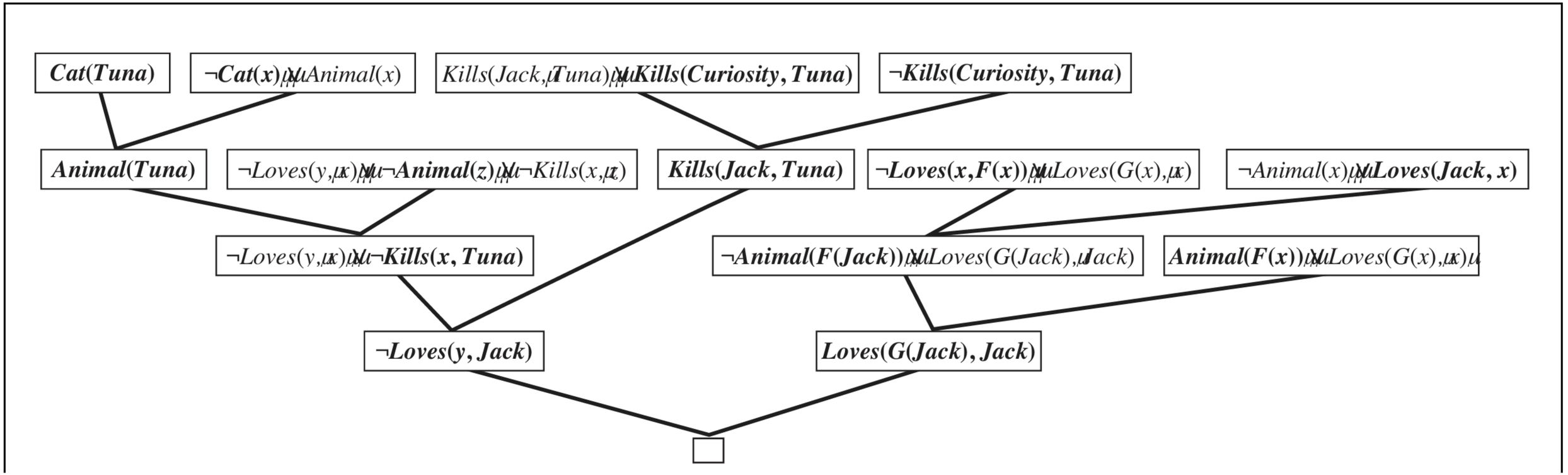
Step 4: Combine the remaining literals

- [Animal(F(x)) ∨ ¬Kills(G(x), x)]

This is the **resolvent**.

1.Everyone who loves all animals is loved by someone.
2.Anyone who kills an animal is loved by no one.
3.Jack loves all animals.
4.Either Jack or Curiosity killed the cat, who is named Tuna.
5.Tuna is a cat.
6.Every cat is an animal.

7.Did Curiosity kill the cat? → this is the query.



Russel and norvig, page 349

Proof using Resolution Refutation in FOL

We want to check if `Criminal(West)` can be inferred from the knowledge base (KB).

KB:

1. ¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x, y, z) ∨ ¬Hostile(z) ∨ Criminal(x)
2. ¬Missile(x) ∨ ¬Owns(Nono, x) ∨ Sells(West, x, Nono)
3. ¬Enemy(x, America) ∨ Hostile(x)
4. ¬Missile(x) ∨ Weapon(x)
5. Owns(Nono, M1)
6. Missile(M1)
7. American(West)
8. Enemy(Nono, America)

9. ¬Criminal(West) ← **negated goal**

Proof using Resolution Refutation in FOL

Step 2: Start resolving clauses

2a: Resolve clause 1 and 9

- Clause 1: ¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x, y, z) ∨ ¬Hostile(z) ∨ **Criminal(x)**
- Clause 9: ¬Criminal(West)

Complementary literals: Criminal(x) and ¬Criminal(West)

Unifier θ: {x/West}

Resolvent:

10. ¬American(West) ∨ ¬Weapon(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z)

Proof using Resolution Refutation in FOL

2b: Resolve clause 10 and 7

- Clause 10: ¬American(West) ∨ ¬Weapon(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z)
- Clause 7: American(West)

Complementary literals: ¬American(West) and American(West)

Resolvent:

11. ¬Weapon(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z)

2c: Resolve clause 11 and 4

- Clause 11: ¬Weapon(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z)
- Clause 4: ¬Missile(x) ∨ Weapon(x)

  Complementary literals: Weapon(y) and ¬Weapon(y) **Unifier θ:** {x/y}

Resolvent:

12. ¬Missile(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z)

Proof using Resolution Refutation in FOL

2d: Resolve clause 12 and 6

- Clause 12: ¬Missile(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z)
- Clause 6: Missile(M1)

Complementary literals: ¬Missile(y) and Missile(M1)
Unifier θ: {y/M1} **Resolvent:**
13. ¬Sells(West, M1, z) ∨ ¬Hostile(z)

2e: Resolve clause 13 and 2

- Clause 13: ¬Sells(West, M1, z) ∨ ¬Hostile(z)
- Clause 2: ¬Missile(x) ∨ ¬Owns(Nono, x) ∨ Sells(West, x, Nono)
- **Complementary literals:** Sells(West, M1, z) and Sells(West, x, Nono) **Unifier θ:** {x/M1, z/Nono}

Resolvent:
14. ¬Missile(M1) ∨ ¬Owns(Nono, M1) ∨ ¬Hostile(Nono)

Proof using Resolution Refutation in FOL

2f: Resolve clause 14 and 6

- Clause 14: ¬Missile(M1) ∨ ¬Owns(Nono, M1) ∨ ¬Hostile(Nono)
- Clause 6: Missile(M1)

Complementary literals: ¬Missile(M1) and Missile(M1)

Resolvent:

15. ¬Owns(Nono, M1) ∨ ¬Hostile(Nono)

2g: Resolve clause 15 and 5

- Clause 15: ¬Owns(Nono, M1) ∨ ¬Hostile(Nono)
- Clause 5: Owns(Nono, M1)

Complementary literals: ¬Owns(Nono, M1) and Owns(Nono, M1)

Resolvent:

16. ¬Hostile(Nono)

Proof using Resolution Refutation in FOL

2h: Resolve clause 16 and 3

- Clause 16: ¬Hostile(Nono)
- Clause 3: ¬Enemy(x, America) ∨ Hostile(x)

Complementary literals: Hostile(x) and ¬Hostile(Nono)
Unifier θ: {x/Nono}
Resolvent:
17. ¬Enemy(Nono, America)

2i: Resolve clause 17 and 8

- Clause 17: ¬Enemy(Nono, America)
- Clause 8: Enemy(Nono, America)

Complementary literals: ¬Enemy(Nono, America) and Enemy(Nono, America)
Resolvent: empty clause

Step 3: Conclusion

- We derived the **empty clause**, which means a contradiction is found.
- Therefore, the goal `Criminal(West)` **can be inferred** from the KB.